



Red Hat OpenShift AI Self-Managed 3.4

Working with machine learning features

Store, manage, and serve features to machine learning models with Feature Store.

Red Hat OpenShift AI Self-Managed 3.4 Working with machine learning features

Store, manage, and serve features to machine learning models with Feature Store.

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack[®] Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

Feature Store provides a framework for storing, managing, and serving features to machine learning models by using your existing infrastructure and data stores.

Table of Contents

PREFACE	4
CHAPTER 1. OVERVIEW OF MACHINE LEARNING FEATURES AND FEATURE STORE	5
1.1. AUDIENCE FOR FEATURE STORE	5
1.2. OVERVIEW OF MACHINE LEARNING FEATURES	5
1.3. OVERVIEW OF FEATURE STORE	6
1.4. FEATURE STORE WORKFLOW	8
1.5. SETTING UP THE FEATURE STORE USER INTERFACE FOR INITIAL USE	8
1.6. ADDITIONAL RESOURCES	9
CHAPTER 2. CONFIGURING FEATURE STORE	11
2.1. SETTING UP FEATURE STORE	11
2.1.1. Before you begin	11
2.1.2. Enabling the Feature Store component	13
2.1.3. Creating a Feature Store instance in a project	14
2.1.4. Configuring and managing Role Based Access Control	17
2.1.5. Adding feature definitions and initializing your Feature Store instance	19
2.1.5.1. Specifying files to ignore	20
2.1.6. Viewing Feature Store objects in the web-based UI	20
2.2. CUSTOMIZING YOUR FEATURE STORE CONFIGURATION	22
2.2.1. Configuring an offline store	23
2.2.2. Configuring an online store	24
2.2.3. Configuring the feature registry	25
2.2.4. Example PVC configuration	27
2.2.5. Editing an existing Feature Store instance	28
2.2.6. Feature server high availability and autoscaling on Kubernetes	28
2.2.6.1. Single-replica limitations for the Feature Store Operator	29
2.2.6.2. Scaling options for production workloads	29
2.2.6.3. High availability features	29
2.2.7. Configuring static replicas for feature servers	29
2.2.8. Configuring a horizontal pod autoscaler for feature servers	30
2.2.9. Customizing feature server high availability	32
2.2.10. Feature Store monitoring reference	34
2.2.10.1. Enabling metrics	34
2.2.10.2. Python feature server available metrics	34
2.3. ENABLING OPENID CONNECT AUTHENTICATION FOR FEATURE STORE	35
2.3.1. OpenID Connect authentication for Feature Store	36
2.3.2. Configuring OpenID Connect authentication for Feature Store	36
2.3.3. Accessing Feature Store data by using a single sign-on	38
CHAPTER 3. DEFINING MACHINE LEARNING FEATURES	40
3.1. SETTING UP YOUR WORKING ENVIRONMENT	40
3.2. ABOUT FEATURE DEFINITIONS	41
3.3. UNDERSTANDING FEATURE STORE DATA TYPES	41
3.3.1. Data type categories	41
3.3.2. Choosing data types	42
3.4. FEATURE STORE DATA TYPE REFERENCE	42
3.4.1. Primitive data types	42
3.4.2. Array data types	43
3.4.3. JSON data types	43
3.4.3.1. Backend support for JSON data types	44
3.4.4. Map data types	44

3.4.4.1. Backend support for map data types	45
3.4.4.2. Map type usage examples	45
3.4.5. Set data types	46
3.4.6. Struct data types	47
3.4.6.1. Backend support for struct data types	47
3.4.7. Complete feature view example	47
3.5. SPECIFYING THE DATA SOURCE FOR FEATURES	49
3.6. ABOUT ORGANIZING FEATURES BY USING ENTITIES	50
3.7. CREATING FEATURE VIEWS	50
CHAPTER 4. RETRIEVING FEATURES FOR MODEL TRAINING	53
4.1. RETRIEVING DATA SCIENCE FEATURES	53
CHAPTER 5. FEATURE STORE INTEGRATION WITH WORKBENCHES	54
5.1. CONNECTING A WORKBENCH TO A FEATURE STORE	54
5.2. VERIFYING ACCESS CONTROL TO YOUR FEATURE STORE INSTANCES	55
5.3. FEATURE STORE CONFIGURATION REFERENCE	56
CHAPTER 6. UNDERSTANDING COMPUTE ENGINES IN FEATURE STORE	57
6.1. USING COMPUTE ENGINES IN FEATURE STORE	57
6.2. UNDERSTANDING THE RAY COMPUTE ENGINE IN FEATURE STORE	59
6.3. GETTING STARTED USING THE RAY TEMPLATE	59
6.4. CONFIGURING RAY IN YOUR FEATURE STORE YAML FILE	60
6.5. UNDERSTANDING RAY MODE DETECTION PRECEDENCE IN FEATURE STORE	62
6.6. USING RAY DIRECTED ACYCLIC GRAPH NODE TYPES IN FEATURE STORE	63
6.7. USING RAY JOIN STRATEGIES IN FEATURE STORE	64
6.8. UNDERSTANDING RAY PERFORMANCE OPTIMIZATION FOR FEATURE STORE	64
6.8.1. Understanding Ray monitoring and metrics in Feature Store	65
6.9. UNDERSTANDING THE SPARK COMPUTE ENGINE IN FEATURE STORE	65
6.10. CONFIGURING SPARK IN YOUR FEATURE STORE YAML FILE	65
6.11. REFERENCE MATERIAL FOR INTEGRATING RAY WITH OTHER COMPONENTS IN FEATURE STORE	67
CHAPTER 7. FEATURE STORE COMMAND LINE INTERFACE REFERENCE	69
7.1. FEATURE STORE GLOBAL COMMAND	69
7.2. FEATURE STORE COMMAND LINE INTERFACE OPTIONS	69
7.3. FEATURE STORE APPLY COMMAND	70
7.4. FEATURE STORE CONFIGURATION COMMAND	71
7.5. FEATURE STORE DELETE COMMAND	71
7.6. FEATURE STORE ENTITIES LIST COMMAND	72
7.7. FEATURE STORE FEATURE VIEWS COMMAND	72
7.8. FEATURE STORE INIT COMMAND	72
7.9. FEATURE STORE MATERIALIZE COMMAND	73
7.10. FEATURE STORE MATERIALIZE-INCREMENTAL COMMAND	74

PREFACE

Feature Store provides an interface between machine learning models and data.

CHAPTER 1. OVERVIEW OF MACHINE LEARNING FEATURES AND FEATURE STORE

A machine learning (ML) feature is a measurable property or attribute within a dataset that a machine learning model can analyze to learn patterns and make decisions. Examples of features include a customer's purchase history, demographic data like age and location, weather conditions, and financial market data. You can use these features to train models for tasks such as personalized product recommendations, fraud detection, and predictive maintenance.

Feature Store is a Red Hat OpenShift AI component that provides a centralized repository that stores, manages, and serves machine learning features for both training and inference purposes.

1.1. AUDIENCE FOR FEATURE STORE

The target audience for Feature Store is ML platform and MLOps teams with DevOps experience in deploying real-time models to production. Feature Store also helps these teams build a feature platform that improves collaboration between data engineers, software engineers, machine learning engineers, and data scientists.

For Data Scientists

Feature Store is a tool where you can define, store, and retrieve your features for both model development and model deployment. By using Feature Store, you can focus on what you do best: build features that power your AI/ML models and maximize the value of your data.

For MLOps Engineers

Feature Store is a library that connects your existing infrastructure, such as online database, application server, microservice, analytical database, and orchestration tooling. By using Feature Store, you can focus on maintaining a resilient system, instead of implementing features for data scientists.

For Data Engineers

Feature Store provides a centralized catalog for storing feature definitions, allowing you to maintain a single source of truth for feature data. It provides the abstraction for reading and writing to many different types of offline and online data stores. Using the provided Python SDK or the feature server service, you can write data to the online and offline stores and then read out that data in either batch scenarios for model training or low-latency online scenarios for model inference.

For AI Engineers

Feature Store provides a platform designed to scale your AI applications by enabling seamless integration of richer data and facilitating fine-tuning. With Feature Store, you can optimize the performance of your AI models while ensuring a scalable and efficient data pipeline.

1.2. OVERVIEW OF MACHINE LEARNING FEATURES

In machine learning, a *feature*, also referred to as a field, is an individual measurable property. A feature is used as an input signal to a predictive model. For example, if a bank's loan department is trying to predict whether an applicant should be approved for a loan, a useful feature might be whether they have filed for bankruptcy in the past or how much credit card debt they currently carry.

Table 1.1. A feature represents a column in a data table

customer_id	avg_cc_balance	credit_score	bankruptcy
1005	500.00	730	0

982	20000.00	570	2
1001	1400.00	600	0

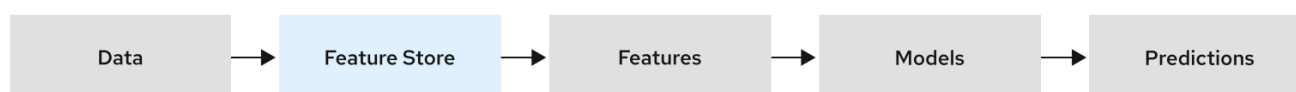
Features are prepared data that help machine learning models understand patterns in the world. Feature engineering is the process of selecting, manipulating, and transforming raw data into features that can be used in supervised learning. As shown in the table, a feature refers to an entire column in a dataset, for example, **credit_score**. A feature value refers to a single value in a feature column, such as **730**.

1.3. OVERVIEW OF FEATURE STORE

Feature Store is an OpenShift AI component that provides an interface between models and data. It is based on the [Feast](#) open source project. Feature Store provides a framework for storing, managing, and serving features to machine learning models by using your existing infrastructure and data stores. It facilitates the retrieval of feature data from different data sources to generate and manage features by providing unified feature management capabilities.

The following figure shows where Feature Store fits in the ML workflow. In an ML workflow, features are inputs to ML models. The ML workflow starts with many types of relevant data, such as transactional data, customer references, and product data. The data comes from a variety of databases and data sources. From this data, ML engineers use Feature Store to curate features. The features are input to models and the models can then use the data from the features to make predictions.

Figure 1.1. Feature Store in the ML workflow



Feature Store is a machine learning data system that provides the following capabilities:

- Runs data pipelines that transform raw data into feature values
- Stores and manages feature data
- Serves feature data consistently for training and inference purposes
- Manages features consistently across offline and online environments
- Powers one model or thousands simultaneously with fresh, reusable features, on demand

Feature Store is a centralized hub for storing, processing, and accessing commonly-used features that enables users in your ML organization to collaborate. When you register a feature in a Feature Store, it becomes available for immediate reuse by other models across your organization. The Feature Store registry reduces duplication of data engineering efforts and allows new ML projects to bootstrap with a library of curated, production-ready features.

Feature Store provides consistency in model training and inference, promotes collaboration and usability across multiple projects, monitors lineage and versioning of models for data drifts, leaks, and training skews, and seamlessly integrates with other MLOps tools. Feature Store remotely manages data

stored in other systems, such as BigQuery, Snowflake, DynamoDB, and Redis, to make features consistently available at training / serving time.

Feature Store performs the following tasks:

- Stores features in offline and online stores
- Registers features in the registry for sharing
- Serves features to ML models

ML platform teams use Feature Store to store and serve features consistently for offline training, such as batch-scoring, and online real-time model inference.

Feature Store consists of the following key components:

Registry

A central catalog of all feature definitions and their related metadata. It allows ML engineers and data scientists to search, discover, and collaborate on new features. The registry exposes methods to apply, list, retrieve, and delete features.

Offline Store

The data store that contains historical data for scale-out batch scoring or model training. The offline store persists batch data that has been ingested into Feature Store. This data is used for producing training datasets. Examples of offline stores include Dask, Snowflake, BigQuery, Redshift, and DuckDB.

Online Store

The data store that is used for low-latency feature retrieval. The online store is used for real-time inference. Examples of online stores include Redis, GCP Datastore, and DynamoDB.

Server

A feature server that serves pre-computed features online. There are three Feature Store servers:

- **The online feature server** - A Python feature server that is an HTTP endpoint that serves features with JSON I/O. You can write and read features from the online store using any programming language that can make HTTP requests.
- **The offline feature server** - An Apache Arrow Flight Server that uses the gRPC communication protocol to exchange data. This server wraps calls to existing offline store implementations and exposes interfaces as Arrow Flight endpoints.
- **The registry server** - A server that uses the gRPC communication protocol to exchange data. You can communicate with the server using any programming language that can make gRPC requests.

UI

A web-based graphical user interface (UI) for viewing all the Feature Store objects and their relationships with each other.

Feature Store provides the following software capabilities:

- A Python SDK for programmatically defining features and data sources
- A Python SDK for reading and writing features to offline and online data stores

- An optional feature server for reading and writing features (useful for non-python languages) by using APIs
- A web-based UI for viewing and exploring information about features defined in the project
- A command line interface (CLI) for viewing and updating feature information

1.4. FEATURE STORE WORKFLOW

The Feature Store workflow involves the following tasks OpenShift cluster administrators, and machine learning (ML) engineers or data scientists:

Note: This Feature Store workflow describes a local implementation that is available in this Technology Preview release.

Cluster administrator

Installs and configures Feature Store, as described in *Chapter 2. Configuring Feature Store* :

1. Installs OpenShift AI.
2. Enables the Feature Store component by using the Feature Store operator.
3. Creates a project.
4. In the project, creates a Feature Store instance by using a **feast.yaml** file that specifies the offline and online stores.
5. Sets up Feature Store so that ML Engineers and data scientists can push and retrieve features to use for model training and inference.

ML Engineer or data scientist

- Prepares features, as described in *Chapter 3: Defining features* :
 1. Creates a feature definition file.
 2. Defines the data sources and other Feature Store objects.
 3. Makes features available for real-time inference.
- Prepares features for model training and real-time inference, as described in *Chapter 4. Retrieving features for model training*:
 1. Makes features available to models.
 2. Uses **feast** Python APIs to retrieve features for model training and inference.

1.5. SETTING UP THE FEATURE STORE USER INTERFACE FOR INITIAL USE

You can use a web-based user interface to simplify and accelerate the creation of model development features. This visual interface helps you explore and understand your Feature Store.

You can use the Feature Store UI to access a centralized catalog of features and metadata, such as transformation logic and materialization job status. You can also view features, manage entities, and use lineage and search capabilities.

You must enable the Feature Store UI before you can use it.

Prerequisites

- You have Administrator access.
- You have enabled the Feast Operator.
- You have created a Feature Store CRD, as described in *Creating a Feature Store instance in a project*.
- Your REST API server is running.

Procedure

1. Log in to the OpenShift AI Dashboard and click the **Feature Store** tab on the left navigation.
2. On the **Overview** page, click the **create FeatureStore** button.
3. Add YAML definitions to enable the user interface.
4. Edit the following label to enable the UI:
 - a. **Label:**
 - b. **feature-store-ui: enabled**
 - c. This creates a pod and initiates the service registry.
5. Click the options icon (⋮) and choose **Start job**.
6. Click on the **Jobs** tab and then **Logs** on the left navigation, to confirm that the CronJob is running.
7. Navigate to the **Feature views** tab on the left navigation, and you will be able to see your new UI.

Verification

Click **Overview**. If your Feature Store user interface was created, you see the following cards: * Entities * Data sources * Datasets * Features * Feature views * Feature services

Additional resources

[Configuring the feature registry](#)

1.6. ADDITIONAL RESOURCES

- For example Feature Store CRD configurations, see the [Feast Operator configuration samples](#).
- For details about the Feast CRD APIs, see the [Feast API documentation](#).
- For information on how to implement machine learning features, see the [Feast documentation](#).

- For end-to-end use case examples of how Feature Store can benefit your AI/ML workflows, see [Feast Getting Started: Use Cases](#).

CHAPTER 2. CONFIGURING FEATURE STORE

As a cluster administrator, you can install and manage Feature Store as a component in the Red Hat OpenShift AI Operator configuration.

2.1. SETTING UP FEATURE STORE

As a cluster administrator, you must complete the following tasks to set up Feature Store:

1. Enable the Feature Store component.
2. Create a project and add a Feature Store instance.
3. Initialize the Feature Store instance.
4. Set up Feature Store so that ML Engineers and data scientists can push and retrieve features to use for model training and inference.

2.1.1. Before you begin

Before you implement Feature Store in your machine learning workflow, you must have the following information:

Knowledge of your data and use case

You must know your use case and your raw underlying data so that you can identify the properties or attributes that you want to define as features. For example, if you are developing machine learning (ML) models that detect possible credit card fraud transactions, you would identify data such as purchase history, transaction location, transaction frequency, or credit limit.

With Feature Store, you define each of those attributes as a feature. You group features that share a conceptual link or relationship together to define an entity. You define entities to map to the domain of your use case. Not all features must be in an entity.

Knowledge of your data source

You must know the source of the raw data that you want to use in your ML workflow. When you configure the Feature Store online and offline stores and the feature registry, you must specify an environment that is compatible with the data source. Also, when you define features, you must specify the data source for the features.

Feature Store uses a time-series data model to represent data. This data model is used to interpret feature data in data sources in order to build training datasets or materialize features into an online store.

You can connect to the following types of data sources:

Batch data source

A method of collecting and processing data in discrete chunks or batches, rather than continuously streaming it. This approach is commonly used for large datasets or when real-time processing is not essential. In a data processing context, a batch data source defines the connection to the data-at-rest source, allowing you to access and process data in batches. Examples of batch data sources include data warehouses (for example, BigQuery, Snowflake, and Redshift) or data lakes (for example, S3 and GCS). Typically, you define a batch data source when you configure the Feature Store offline store.

Stream data source

The origin of data that is continuously flowing or emitted for online, real-time processing. Feature Store does not have native streaming integrations, but it facilitates push sources that allow you to push features into Feature Store. You can use Feature Store for training or batch scoring (offline), for real-time feature serving (online), or for both. Typically, you define a stream data source when you configure the Feature Store online store.

You can use the following data sources with Feature Store:

Data sources for online stores

- SQLite
- Snowflake
- Redis
- Dragonfly
- IKV
- Datastore
- DynamoDB
- Bigtable
- PostgreSQL
- Cassandra + Astra DB
- Couchbase
- MySQL
- Hazelcast
- ScyllaDB
- Remote
- SingleStore

For details on how to configure these online stores, see the [Feast reference documentation for online stores](#).

Data sources for offline stores

- Dask
- Snowflake
- BigQuery
- Redshift
- DuckDB

An offline store is an interface for working with historical time-series feature values that are stored in data sources. Each offline store implementation is designed to work only with the corresponding data source.

Offline stores are useful for the following purposes:

- To build training datasets from time-series features.
- To materialize (load) features into an online store to serve those features at low-latency in a production setting.

You can use only a single offline store at a time. Offline stores are not compatible with all data sources; for example, the BigQuery offline store cannot be used to query a file-based data source.

For details on how to configure these offline stores, see the [Feast reference documentation for offline stores](#).

Data sources for the feature registry

- Local
- S3
- GCS
- SQL
- Snowflake

For details on how to configure these registry options, see the [Feast reference documentation for the registry](#).

2.1.2. Enabling the Feature Store component

To allow the ML engineers and data scientists in your organization to work with machine learning features, you must enable the Feature Store component in Red Hat OpenShift AI.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed OpenShift AI.

Procedure

1. In the OpenShift console, click **Ecosystem → Installed Operators**.
2. Click the **Red Hat OpenShift AI** Operator.
3. Click the **Data Science Cluster** tab.
4. Click the default instance name (for example, **default-dsc**) to open the instance details page.
5. Click the **YAML** tab.
6. Edit the **spec:components** section. For the **feastoperator** component, set the **managementState** field to **Managed**:

```
spec:
  components:
    feastoperator:
      managementState: Managed
```

7. Click **Save**.

Verification

Check the status of the **feast-operator-controller-manager-`<pod-id>`** pod:

1. Click **Workloads → Deployments**.
2. From the **Project** list, select **redhat-ods-applications**.
3. Search for the **feast-operator-controller-manager** deployment.
4. Click the **feast-operator-controller-manager** deployment name to open the deployment details page.
5. Click the **Pods** tab.
6. View the pod status.

When the status of the **feast-operator-controller-manager-`<pod-id>`** pod is **Running**, Feature Store is enabled.

Next Step

- Create a Feature Store instance in a project.

2.1.3. Creating a Feature Store instance in a project

You can add an instance of Feature Store to a project by creating a custom resource definition (CRD) in the OpenShift console.

The following example shows the minimum requirements for a Feature Store CR YAML file:

```
apiVersion: feast.dev/v1alpha1
kind: FeatureStore
metadata:
  name: sample
spec:
  feastProject: my_feast_project
```

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have enabled the Feature Store component, as described in *Enabling the Feature Store component*.
- You have set up your database infrastructure for the online store, offline store, and registry.

For an example of setting up and running PostgreSQL (for the registry) and Redis (for the online store), see the Feature Store Operator quick start example: <https://github.com/feast-dev/feast/tree/stable/examples/operator-quickstart>.

- You have created a project, as described in [Creating a project](#). In the following procedure, **my-project** is the name of the project.

Procedure

1. In the OpenShift console, click the **Quick Create** () icon and then click the **Import YAML** option.
2. Verify that your project is the selected project.
3. Copy the following code and paste it into the YAML editor:

```
apiVersion: feast.dev/v1alpha1
kind: FeatureStore
metadata:
  name: sample-git
spec:
  feastProject: credit_scoring_local
  feastProjectDir:
    git:
      url: https://github.com/feast-dev/feast-credit-score-local-tutorial
      ref: 598a270
```

The **spec.feastProjectDir** references a Feature Store project that is in the Git repository for a Credit Store tutorial.

4. Optionally, change the **metadata.name** for the Feature Store instance.
5. Optionally, edit **feastProject**, which is the namespace for organizing your Feature Store instance. Note that this project is not the OpenShift AI project.
6. Click **Create**.

When you create the Feature Store CR in OpenShift, Feature Store starts a remote online feature server, and configures a default registry and an offline store with the local provider.

A *provider* is a customizable interface that provides default Feature Store components, such as the registry, offline store, and online store, that target a specific environment, ensuring that these components can work together seamlessly. The local provider uses the following default settings:

- **Registry:** A SQL registry or local file
- **Offline store:** A Parquet file
- **Online store:** SQLite

Verification

1. In the OpenShift console, select **Workloads → Pods**.
2. Make sure that your project (for example, my-project) is selected.

3. Find the pod that has the **feast-** prefix, followed by the **metadata.name** that you specified in the CRD configuration, for example, **sample-git**.
4. Verify that the pod status is **Running**.
5. Click the **feast** pod and then select **Pod details**.
6. Scroll down to see the online container. This container is the deployment for the online server. It makes the feature server REST API available in the OpenShift cluster.
7. Scroll up and then click **Terminal**.
8. Run the following command to verify that the **feast** CLI is installed correctly:

```
$ feast --help
```

9. To view the files for the Feature Store project, enter the following command:

```
$ ls -la
```

You should see output similar to the following:

```
.
..
data
example_repo.py
feature_store.yaml
__init__.py
__pycache__
test_workflow.py
```

10. To view the **feature_store.yaml** configuration file, enter the following command:

```
$ cat feature_store.yaml
```

You should see output similar to the following:

```
project: my_feast_project
provider: local
online_store:
  path: /feast-data/online_store.db
  type: sqlite
registry:
  path: /feast-data/registry.db
  registry_type: file
auth:
  type: no_auth
entity_key_serialization_version: 3
```

The **feature_store.yaml** file defines the following components:

- **project** – The namespace for the Feature Store instance. Note that this project refers to the feature project rather than the OpenShift AI project.

- **provider** – The environment in which Feature Store deploys and operates.
- **registry** – The location of the feature registry.
- **online_store** – The location of the online store.
- **auth** – The type of authentication and authorization (**no_auth**, **kubernetes**, or **oidc**)
- **entity_key_serialization_version** – Specifies the serialization scheme that Feature Store uses when writing data to the online store.

NOTE: Although the **offline_store** location is not included in the **feature_store.yaml** file, the Feature Store instance uses a DASK file-based offline store. In the **feature_store.yaml** file, the registry type is **file** but it uses a simple SQLite database.

Next steps

- Optionally, you can customize the default configurations for the offline store, online store, or registry by editing the YAML configuration for the Feature Store CR, as described in *Customizing your Feature Store configuration*.
- Give your ML engineers and data scientists access to the project so that they can create a workbench. and provide them with a copy of the **feature_store.yaml** file so that they can add it to their workbench IDE, such as Jupyter.

2.1.4. Configuring and managing Role Based Access Control

You can set permissions using Role-Based Access Control (RBAC) to manage user access to Feature Store. This grants access to actions such as creating, reading, updating and deleting namespaces.

Prerequisites

- You have Administrator access.
- You have created a Feature Store instance.



NOTE

For more information, see [What is Kubernetes Role Based Access Control ?](#)

Procedure

1. Open your command line interface (CLI). Deploy the Feature store custom resource by running the following command:

```
kubectl apply -f feature-store-cr.yaml
```

- a. Locate the **Feature Store Custom Resource (CR) YAML file** which is named **feature-store-cr.yaml**. You will see key value pairs. Change the key **type:** to **Kubernetes**:

```
apiVersion: feast.dev/v1alpha1
kind: FeatureStore
metadata:
  name: <feature-store-name>
spec: # ... other configurations ...
```

```
authz:
type: Kubernetes
```

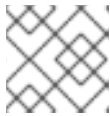
2. Verify that your Feature Store projects were created.

```
kubectl get feast

<project name>
kubectl get configmaps -l feast.dev/service-type=client

<your-project-name> <feast project name> <number of data entries> <time since created>
```

3. Configure data science project permissions. You must create a **permissions.py** file in the Feature Store pod terminal. This file must reside in the **feature_store directory**. You can use a role based policy, a group based policy, combined group namespace policy or read and write permissions.



NOTE

For an example of a **permission.py** file, see the [Feast Operator RBAC with TLS](#).

4. Transfer your local **permissions.py** file to the remote container filesystem. In a Kubernetes/OpenShift environment, you use a command-line tool such as oc OpenShift Command Line Interface or kubectl:

```
`oc/kubectl cp <local-file> <remote-pod>:<remote-path>.`
```

5. Configure and set up the Feature Store Server. If a cron job has been run previously, run feast apply on the online container. Open your command line interface (CLI) and run the following command:

```
`oc create job --from=cronjob/feast-project-name cronjob-manual-$(date +%s) -n <project name>`

`oc exec -it deployments/<feast deployment name> -c online -- feast apply`
```

6. Configure authentication in the OpenShift web console. You have full control over your data science project access. You can grant and revoke access to users/groups instantly.
 - a. Log in to your OpenShift AI or OpenShift Console.
 - b. Navigate to the **Data Science Projects** tab and select the appropriate project.
 - c. Click **Permissions tab > Users Groups**.
 - d. Name your group.
 - e. Under **Permissions**, choose a predefined role, add permissions, and click **Save**.



NOTE

The name of your group must exist in your identity provider. The identity provider is configured at the OpenShift cluster level, outside of the specific project you are working in.

Verification

The deployment pod is running and you see the project details in the Feature Store UI and **Integration** tab.

Additional resources

- [Setting up Kubernetes Authentication](#)

2.1.5. Adding feature definitions and initializing your Feature Store instance

Initialize the Feature Store instance to start using it.

When you initialize the Feature Store instance, Feature Store completes the following tasks:

- Scans the Python files in your feature repository and finds all Feature Store object definitions, such as feature views, entities, and data sources.
Note: Feature Store reads all Python files recursively, including subdirectories, even if they do not contain feature definitions. For information on identifying Python files, such as imperative scripts that you want Feature Store to ignore, see *Specifying files to ignore*.
- Validates your feature definitions, for example, by checking for uniqueness of features within a feature view.
- Syncs the metadata about objects to the feature registry. If a registry does not exist, Feature Store creates one. The default registry is a simple Protobuf binary file on disk (locally or in an object store).
- Creates or updates all necessary Feature Store infrastructure. The exact infrastructure that Feature Store creates depends on the provider configuration that you have set in **feature_store.yaml**. For example, when you specify **local** as your provider, Feature Store creates the infrastructure on the local cluster.
Note: When you use a cloud provider, such as Google Cloud Platform or Amazon Web Service, the **feast apply** command creates cloud infrastructure that might incur costs for your organization.

Prerequisites

- An ML engineer on your team has given you a Python file that defines features. For more information about how to define features, see *Defining features*.
- If you want to store the feature registry in cloud storage or in a database, you have configured storage for the feature registry. For example, if the provider is GCP, you have created a Cloud Storage bucket for the feature registry.
- You have the **cluster-admin** role in OpenShift.
- You have created a Feature Store instance in your project.

Procedure

1. In the OpenShift console, select **Workloads** → **Pods**.
2. Make sure that your project is the current project.
3. Click the **feast** pod and then select **Pod details**.

4. Scroll down to see the online container. This container is the deployment for the online server, and it makes the feature server REST API available in the OpenShift cluster.
5. Scroll up and then click **Terminal**.
6. Copy the feature definition (**.py**) file to your Feature Store directory.
7. To create a feature registry and add the feature definitions to the registry, run the following command:

```
feast apply
```

Verification

- You should see output similar to the following that indicates that the features in the feature definition file were successfully added to the registry:

```
Created project credit_scoring_local
Created entity zipcode
Created entity dob_ssn
Created feature view zipcode_features
Created feature view credit_history
Created on demand feature view total_debt_calc

Created sqlite table credit_scoring_local_credit_history
Created sqlite table credit_scoring_local_zipcode_features
```

- In the OpenShift console, select **Workloads** → **Deployments** to view the deployment pod.

2.1.5.1. Specifying files to ignore

When you run the **feast apply** command, Feature Store reads all Python files recursively, including Python files in subdirectories, even if the Python files do not contain feature definitions.

If you have Python files, such as imperative scripts, in your registry folder that you want Feature Store to ignore when you run the **feast apply** command, you should create a **.feastignore** file and add a list of paths to all files that you want Feature Store to ignore.

Example .feastignore file

```
# Ignore virtual environment
venv

# Ignore a specific Python file
scripts/foo.py

# Ignore all Python files directly under scripts directory
scripts/*.py

# Ignore all "foo.py" anywhere under scripts directory
scripts/**/foo.py
```

2.1.6. Viewing Feature Store objects in the web-based UI

You can use the Feature Store Web UI to view all registered features, data sources, entities, and feature services.

Prerequisites

- You can access the OpenShift console.
- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS
- You have enabled the Feature Store component, as described in *Enabling the Feature Store component*.
- You have created a Feature Store CRD, as described in *Creating a Feature Store instance in a project*.

Procedure

1. In the OpenShift console, select **Administration** → **CustomResourceDefinitions**.
2. To filter the list, in the **Search by Name** field, enter **feature**.
3. Click the **FeatureStore** CRD and then click **Instances**.
4. Click the name of the instance that corresponds to the metadata name you specified when you created the Feature Store instance.
5. Edit the YAML to include a reference to **services.ui** in the **spec** section, as shown in the following example:

```
spec:
  feastProject: credit_scoring_local
  feastProjectDir:
    git:
      ref: 598a270
      url: 'https://github.com/feast-dev/feast-credit-score-local-tutorial'
  services:
    ui: {}
```

6. Click **Save** and then click **Reload**.
The Feature Store Operator starts a container for the web-based Feature Store UI and creates an OpenShift route that provides the URL so that you can access it.
7. In the OpenShift console, select **Workloads** → **Pods**.
8. Make sure that your project (for example, **my-project**) is selected.
You should see a deployment for the web-based UI. Note that OpenShift enables TLS by default at runtime.
9. To populate the web-based UI with the objects in your Feature Store instance:
 - a. In the OpenShift console, select **Workloads** → **Pods**.

- b. Make sure that your project (for example, **my-project**) is selected.
- c. Click the **feast** pod and then select **Pod details**.
- d. Click **Terminal**.
- e. To update the Feature Store instance, enter the following command:

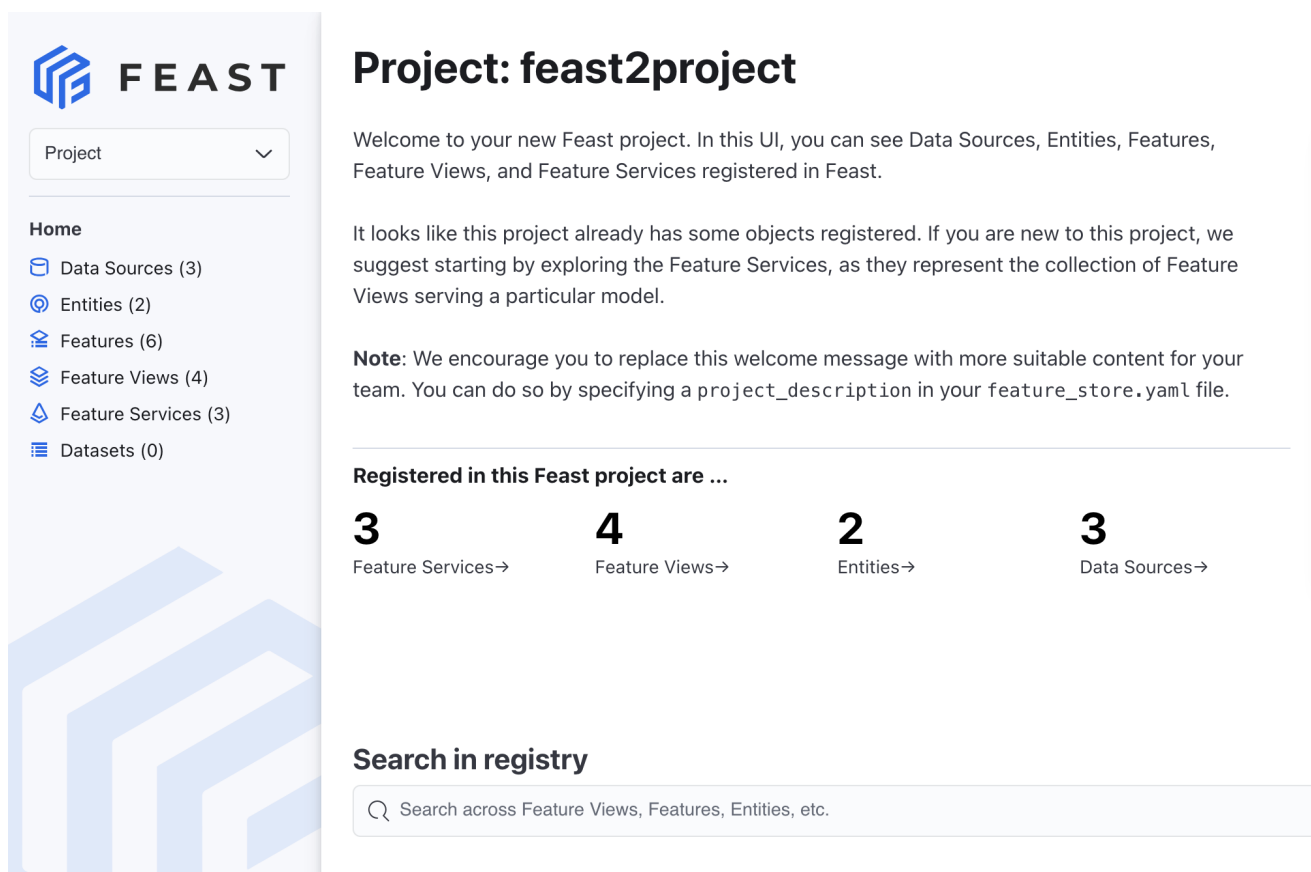
```
feast apply
```

10. To find the URL for the Feature Store UI, in the OpenShift console, click **Networking → Routes**. In the row for the Feature Store UI, for example **feast-sample-ui**, the URL is in the **Location** column.
11. Click the URL link to open it in your default web browser.

Verification

The Feature Store Web UI is displayed and shows the feature objects in your project as shown in the following figure:

Figure 2.1. The Feature Store Web UI



2.2. CUSTOMIZING YOUR FEATURE STORE CONFIGURATION

Optionally, you can apply the following configurations to your Feature Store instance:

- Configure an offline store
- Configure an online store

- Configure the feature registry
- Configure persistent volume claims (PVCs)
- Configure role-based access control (RBAC)

The examples in the following sections describe how to customize a feature store instance by creating a new custom resource definition (CRD). Alternatively, you can customize an existing feature instance as described in *Editing an existing feature store instance*.

For more information about how you can customize your feature store configuration, see the [Feast API documentation](#).

2.2.1. Configuring an offline store

When you create a Feature Store instance that uses the minimal configuration, by default, Feature Store uses a SQLite file-based store for the offline store.

The example in the following procedure shows how to configure DuckDB for the offline store.

You can configure other offline stores, such as Snowflake, BigQuery, Redshift, as detailed in the [Feast reference documentation for offline stores](#).



NOTE

The example code in the following procedure requires that you edit it with values that are specific to your use case.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have enabled the Feature Store component, as described in *Enabling the Feature Store component*.
- You have created a project, as described in [Creating a project](#). In the following procedure, **my-project** is the name of the project.
- Your project includes an existing secret that provides credentials for accessing the database that you want to use for the offline store. The example in the following procedure requires that you have configured DuckDB.

Procedure

1. In the OpenShift console, click the **Quick Create** () icon and then click the **Import YAML** option.
2. Verify that your project is the selected project.
3. Copy the following code and paste it into the YAML editor:

```
apiVersion: feast.dev/v1alpha1
kind: FeatureStore
metadata:
```

```

name: sample-db-persistence
spec:
  feastProject: my_project
  services:
    offlineStore:
      persistence:
        file:
          type: duckdb

```

4. Edit the **services.offlineStore** section to specify values specific to your use case.
5. Click **Create**.

Verification

1. In the OpenShift console, select **Workloads → Pods**.
2. Make sure that your project (for example, **my-project**) is selected.
3. Find the pod that has the **feast-** prefix, followed by the metadata name that you specified in the CRD configuration, for example, **feast-sample-db-persistence**.
4. Verify that the status is **Running**.

2.2.2. Configuring an online store

When you create a Feature Store instance using the minimal configuration, by default, the online store is a SQLite database.

The example in the following procedure shows how to configure a PostgreSQL database for the online store.

You can configure other online stores, such as Snowflake, Redis, and DynamoDB, as detailed in the [Feast reference documentation for online stores](#).



NOTE

The example code in the following procedure requires that you edit it with values that are specific to your use case.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have enabled the Feature Store component, as described in *Enabling the Feature Store component*.
- You have created a project, as described in [Creating a project](#). In the following procedure, **my-project** is the name of the project.
- Your project includes an existing secret that provides credentials for accessing the database that you want to use for the online store. The example in the following procedure requires that you have configured a PostgreSQL database.

Procedure

1. In the OpenShift console, click the **Quick Create** () icon and then click the **Import YAML** option.
2. Verify that your project is the selected project.
3. Copy the following code and paste it into the YAML editor:

```
apiVersion: feast.dev/v1alpha1
kind: FeatureStore
metadata:
  name: sample-db-persistence
spec:
  feastProject: my_project
  services:
    onlineStore:
      persistence:
        store:
          type: postgres
          secretRef:
            name: feast-data-stores
```

4. Edit the **services.onlineStore** section to specify values that are specific to your use case.
5. Click **Create**.

Verification

1. In the OpenShift console, select **Workloads → Pods**.
2. Make sure that your project (for example, **my-project**) is selected.
3. Find the pod that has the **feast-** prefix, followed by the metadata name that you specified in the CRD configuration, for example, **feast-sample-db-persistence**.
4. Verify that the status is **Running**.

2.2.3. Configuring the feature registry

By default, when you create a feature instance using the minimal configuration, the registry is a simple SQLite database.

The example in the following procedure shows how to configure an S3 registry.

You can configure other types of registries, such as GCS, SQL, Snowflake, as detailed in the [Feast reference documentation for registries](#).



NOTE

The example code in the following procedure requires that you edit it with values that are specific to your use case.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.

- You have enabled the Feature Store component, as described in *Enabling the Feature Store component*.
- You have created a project, as described in [Creating a project](#). In the following procedure, **my-project** is the name of the project.
- Your project includes an existing secret that provides credentials for accessing the database that you want to use for the registry. The example in the following procedure requires that you have configured S3.

Procedure

1. In the OpenShift console, click the **Quick Create** () icon and then click the **Import YAML** option.
2. Verify that your project is the selected project.
3. Copy the following code and paste it into the YAML editor:

```
apiVersion: feast.dev/v1alpha1
kind: FeatureStore
metadata:
  name: sample-s3-registry
spec:
  feastProject: my_project
  services:
    registry:
      local:
        server:
          restAPI: true
        persistence:
          file:
            path: s3://bucket/registry.db
            s3_additional_kwargs:
              ServerSideEncryption: AES256
              ACL: bucket-owner-full-control
              CacheControl: max-age=3600
```

4. Edit the **services.registry** section to specify values that are specific to your use case.
5. Click **Create**. You have now configured your registry service and enabled the REST APIs.

Verification

1. In the OpenShift console, select **Workloads → Pods**.
2. Make sure that your project (for example, **my-project**) is selected.
3. Find the pod that has the **feast-** prefix, followed by the metadata name that you specified in the CRD configuration, for example, **sample-s3-registry**.
4. Click the feast pod and then select **Pod details**.
5. Click **Terminal**.

6. In the Terminal window, enter the following command to view the configuration, including the S3 registry:

```
$ cat feature_store.yaml
```

2.2.4. Example PVC configuration

When you configure the online store, offline store, or registry, you can also configure persistent volume claims (PVCs) as shown in the following Feature Store custom resource definition (CRD) example.



NOTE

The following example code requires that you edit it with values that are specific to your use case.

```
apiVersion: feast.dev/v1alpha1
kind: FeatureStore
metadata:
  name: sample-pvc-persistence
spec:
  feastProject: my_project
  services:
    onlineStore: ❶
      persistence:
        file:
          path: online_store.db
        pvc:
          ref:
            name: online-pvc
            mountPath: /data/online
    offlineStore: ❷
      persistence:
        file:
          type: duckdb
        pvc:
          create:
            storageClassName: standard
          resources:
            requests:
              storage: 5Gi
          mountPath: /data/offline
  registry: ❸
    local:
      persistence:
        file:
          path: registry.db
        pvc:
          create: {}
          mountPath: /data/registry
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: online-pvc
```

```
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
```

- 1 The online store specifies a PVC that must already exist.
- 2 The offline store specifies a storage class name and storage size.
- 3 The registry configuration specifies that the Feature Store Operator creates a PVC with default settings.

2.2.5. Editing an existing Feature Store instance

The examples in this document describe how to customize a Feature Store instance by creating a new custom resource definition (CRD). Alternatively, you can customize an existing feature instance.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have created a Feature Store instance, as described in *Deploying a Feature Store instance in a project*.

Procedure

1. In the OpenShift console, select **Administration** → **CustomResourceDefinitions**.
2. To filter the list, in the **Search by Name** field, enter **feature**.
3. Click the **FeatureStore** CRD and then click **Instances**.
4. Select the instance that you want to edit, and then click **YAML**.
5. In the YAML editor, edit the configuration.
6. Click **Save** and then click **Reload**.

Verification

The Feature Store instance CRD deploys successfully.

2.2.6. Feature server high availability and autoscaling on Kubernetes

You can use horizontal scaling to handle production workloads and ensure high availability for real-time inference and batch scoring operations. You can also configure scaling to meet latency and availability service level agreements (SLAs) for mission-critical applications.



IMPORTANT

To scale horizontally, use database-backed persistence for all enabled services: online store, offline store, and registry. Do not use file-based persistence, such as SQLite, DuckDB, or **registry.db**, with multiple replicas. These backends do not support concurrent access from multiple pods.

2.2.6.1. Single-replica limitations for the Feature Store Operator

The Feature Store Operator deploys a single replica by default. This configuration supports development environments, but it restricts production workloads in the following ways:

- Single point of failure: Pod crashes cause downtime for all feature consumers.
- Limited throughput: A single pod handles a finite number of concurrent requests.
- No elasticity: Traffic spikes from model retraining or batch inference can overwhelm the server.
- Update downtime: The default **Recreate** strategy stops the old pod before starting a new one.

2.2.6.2. Scaling options for production workloads

You can choose from two scaling configurations to meet different production requirements:

Static replicas

Use this configuration to set a fixed number of replicas. This configuration provides high availability and load distribution with a predictable resource footprint. The Operator automatically switches the deployment strategy to **RollingUpdate**, ensuring zero-downtime deployments.

Horizontal pod autoscaling (HPA)

Use this configuration for workloads with variable traffic patterns. Configure the Operator to create and manage a HPA using the **services.scaling.autoscaling** parameter.

2.2.6.3. High availability features

When you enable scaling, the Operator activates two high-availability features to spread pods across failure domains and protect them during disruptions.

Pod anti-affinity

The Operator automatically injects a soft pod anti-affinity rule to spread pods across different nodes. This rule places each replica on a separate node if possible, but does not prevent scheduling if nodes are constrained.

Topology spread constraints

When you configure autoscaling or set more than one replica, the Operator adds a soft zone spread constraint to distribute pods across availability zones whenever possible.

Additional resources

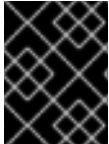
- [Kubernetes Horizontal Pod Autoscaling](#)

2.2.7. Configuring static replicas for feature servers

Set a fixed number of replicas to provide high availability and load distribution with a predictable resource footprint.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have created a Feature Store instance.
- You have configured database-backed persistence for the following enabled services:
 - online store
 - offline store
 - registry



IMPORTANT

Do not use file-based persistence, such as SQLite, DuckDB, or **registry.db**, with multiple replicas. These backends do not support concurrent access from multiple pods.

Procedure

1. Scale the static replicas by running the following command:

```
$ oc scale featurestore/production-featurestore --replicas=3
```

Alternatively, you can set the replicas in the **FeatureStore** custom resource:

```
apiVersion: feast.dev/v1
kind: FeatureStore
metadata:
  name: production-feature-store
spec:
  feastProject: my_project
  replicas: 3
  services:
    onlineStore:
      persistence:
        store:
          type: postgres
          secretRef:
            name: feature-store-data-stores
    registry:
      local:
        persistence:
          store:
            type: sql
            secretRef:
              name: feature-store-data-stores
```

2.2.8. Configuring a horizontal pod autoscaler for feature servers

For workloads with variable traffic patterns, configure the Operator to create and manage a horizontal pod autoscaler (HPA).



NOTE

Manually patching deployments or creating external HPAs bypasses the Operator's reconciliation loop and causes configuration drift.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have created a Feature Store instance.
- You have configured database-backed persistence for all enabled services: online store, offline store, and registry. Do not use file-based persistence, such as SQLite, DuckDB, or **registry.db**, with multiple replicas. These backends do not support concurrent access from multiple pods.

Procedure

1. Configure the HPA by setting the **services.scaling.autoscaling** parameter in the FeatureStore custom resource:

```
apiVersion: feast.dev/v1
kind: FeatureStore
metadata:
  name: autoscaled-feature-store
spec:
  feastProject: my_project
  services:
    scaling:
      autoscaling:
        minReplicas: 2
        maxReplicas: 10
        metrics:
          - type: Resource
            resource:
              name: cpu
            target:
              type: Utilization
              averageUtilization: 70
  podDisruptionBudgets:
    maxUnavailable: 1
  onlineStore:
    persistence:
      store:
        type: postgres
        secretRef:
          name: feature-store-data-stores
  server:
    resources:
      requests:
        cpu: 200m
        memory: 256Mi
      limits:
        cpu: "1"
        memory: 1Gi
  registry:
    local:
```

```

persistence:
  store:
    type: sql
    secretRef:
      name: feature-store-data-stores

```

where:

minReplicas

Specifies the minimum number of replicas.

maxReplicas

Specifies the maximum number of replicas.

metrics

Specifies the metrics to use for autoscaling. If you do not specify custom metrics, the Operator defaults to 80% CPU utilization.

podDisruptionBudgets

Specifies the maximum number of pods that can be unavailable during disruptions.

The following configuration details apply:

+

Mutual exclusivity

Autoscaling is mutually exclusive with **spec.replicas > 1**.

Resource ownership

The Operator creates the HPA as an owned resource and automatically removes it if you remove the autoscaling configuration or delete the FeatureStore custom resource (CR).

Default metrics

If you do not specify custom metrics, the Operator defaults to 80% CPU utilization.

Resilience

To improve resilience, the Operator automatically injects soft pod anti-affinity (node-level) and topology spread (zone-level) constraints.

Verification

- Verify that the Operator created the HorizontalPodAutoscaler resource:

```
$ oc get hpa
```

2.2.9. Customizing feature server high availability

You can customize the high-availability behavior of feature servers by configuring pod anti-affinity rules, topology spread constraints, and pod disruption budgets.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have created a Feature Store instance.

- You have configured scaling by setting **replicas > 1** or enabling autoscaling.

Procedure

1. Customize pod anti-affinity by providing your own affinity configuration in the **FeatureStore** custom resource (CR):

The Operator automatically injects a soft pod anti-affinity rule to spread pods across different nodes. To override this behavior, set the value to

requiredDuringSchedulingIgnoredDuringExecution for strict anti-affinity:

```
affinity:
  podAntiAffinity:
    preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
    podAffinityTerm:
      topologyKey: kubernetes.io/hostname
      labelSelector:
        matchLabels:
          feast.dev/name: my-feature-store
```

2. Customize topology spread constraints to control how pods are distributed across availability zones:

When you configure autoscaling or set **replicas > 1**, the Operator automatically injects a soft zone-spread constraint. This constraint distributes pods across availability zones whenever possible. To override this behavior, provide explicit constraints such as strict **DoNotSchedule**:

```
topologySpreadConstraints:
  - maxSkew: 1
    topologyKey: topology.kubernetes.io/zone
    whenUnsatisfiable: ScheduleAnyway
    labelSelector:
      matchLabels:
        feast.dev/name: my-feature-store
```

To disable this constraint entirely, set **topologySpreadConstraints: []**.

3. Configure a pod disruption budget (PDB) to protect pods during voluntary disruptions, such as node drains and cluster upgrades:

Set exactly one of the **minAvailable** or **maxUnavailable** parameters. The Operator uses Common Expression Language (CEL) validation to enforce this requirement.

```
spec:
  replicas: 3
  services:
    podDisruptionBudgets:
      maxUnavailable: 1
    onlineStore:
```

The Operator creates the PDB only when you enable scaling and automatically removes it when you disable scaling.

Additional resources

- [Kubernetes Pod Disruptions](#)

2.2.10. Feature Store monitoring reference

Feature Store exposes Prometheus metrics for monitoring feature server performance, online store operations, materialization pipelines, and feature freshness. Use these metrics to ensure your feature serving infrastructure meets latency and availability service level agreements for production applications.

2.2.10.1. Enabling metrics

If you run Feast on Kubernetes with the Feast Operator, set the **metrics: true** parameter on the online store server. The operator appends **--metrics** to the serve command and exposes port 8000 as the metrics port on the Service. When the Prometheus Operator is installed on your cluster, the operator automatically creates a service monitor resource for metric discovery. If the **monitoring.coreos.com** API group is not available, service monitor creation is skipped.

```
apiVersion: feast.dev/v1
kind: FeatureStore
metadata:
  name: production-feature-store
spec:
  feastProject: my_project
  services:
    onlineStore:
      server:
        metrics: true
```

2.2.10.2. Python feature server available metrics

Feature Store exposes the following Prometheus metrics:

Metric	Type	Labels	Category	Description
feast_feature_server_cpu_usage	Gauge	—	resource	Process CPU usage %
feast_feature_server_memory_usage	Gauge	—	resource	Process memory usage %
feast_feature_server_request_total	Counter	endpoint, status	request	Total requests per endpoint
feast_feature_server_request_latency_seconds	Histogram	endpoint, feature_count, feature_view_count	request	Request latency with p50/p95/p99 support
feast_online_features_request_total	Counter	—	online_features	Total online feature retrieval requests

Metric	Type	Labels	Category	Description
feast_online_features_entity_count	Histogram	–	online_features	Entity rows per online feature request
feast_feature_server_online_store_read_duration_seconds	Histogram	–	online_features	Online store read phase duration (sync and async)
feast_feature_server_transformation_duration_seconds	Histogram	odfv_name, mode	online_features	ODFV read-path transformation duration (requires track_metrics=True on the ODFV)
feast_feature_server_write_transformation_duration_seconds	Histogram	odfv_name, mode	online_features	ODFV write-path transformation duration (requires track_metrics=True on the ODFV)
feast_push_requests_total	Counter	push_source, mode	push	Push requests by source and mode
feast_materialization_result_total	Counter	feature_view, status	materialization	Materialization runs (success/failure)
feast_materialization_duration_seconds	Histogram	feature_view	materialization	Materialization duration per feature view
feast_feature_freshness_seconds	Gauge	feature_view, project	freshness	Seconds since last materialization

Additional resources

- [Prometheus configuration documentation](#)
- [Prometheus Operator Service Monitor documentation](#)

2.3. ENABLING OPENID CONNECT AUTHENTICATION FOR FEATURE STORE

You can configure OpenID Connect (OIDC) authentication for Feature Store to enforce fine-grained access control by using your existing identity provider. After configuration, you can access Feature Store data through single sign-on without additional authentication.

2.3.1. OpenID Connect authentication for Feature Store

You can use external OpenID Connect (OIDC) providers, such as Keycloak, for centralized identity management in Red Hat OpenShift AI's Feature Store.

Feature Store extracts user attributes from OIDC tokens, including groups and namespaces, to automatically enforce access policies. You can use the following native Feast role-based access control (RBAC) policies with OIDC tokens:

- Group-based policy
- Namespace-based policy
- Combined group and namespace policy

2.3.2. Configuring OpenID Connect authentication for Feature Store

You can configure OpenID Connect (OIDC) authentication for Feature Store to achieve a unified authentication experience with your external identity provider.

Prerequisites

- You have permission to create Feature Store resources in your namespace.
- Your cluster has an OIDC identity provider configured. If you are setting up OIDC for the first time, see [Configuring a central authentication service for an external OIDC identity provider](#) .

Procedure

1. Create a **FeatureStore** custom resource (CR) with OIDC authentication enabled.
When Red Hat OpenShift AI is configured with the centralized Gateway API, the platform automatically provides the OIDC configuration to the Feature Store Operator. You must add the **authz.oidc** section to enable OIDC authentication.

The following YAML example shows where to add the **authz.oidc** section:

```
apiVersion: feast.dev/v1
kind: FeatureStore
metadata:
  name: my-feature-store
spec:
  feastProject: my_project
  authz:
    oidc: {}
```




NOTE

If you configure the OIDC identity provider outside the cluster and it is not integrated through the Gateway API, specify the issuer URL.

The following YAML code shows how to specify the issuer URL:

```
authz:
  oidc:
    issuerUrl: https://keycloak.example.com/realms/my-realm
```

1. Optional: If you need access control by using OIDC client roles, create a **Secret** containing your OIDC client ID and reference it in the **FeatureStore** CR.

The following YAML code shows how to create a **Secret** with your OIDC client ID:

```
apiVersion: v1
kind: Secret
metadata:
  name: feast-oidc-secret
stringData:
  client_id: feast-server
```

2. Reference the **Secret** in the CR:

```
spec:
  authz:
    oidc:
      secretRef:
        name: feast-oidc-secret
```

1. Apply the **FeatureStore** custom resource:

```
$ oc apply -f feature-store.yaml
```

Verification

1. Verify that the **FeatureStore** is ready:

```
$ oc get feast -n <namespace>
```

If the procedure was successful, you will see a ready in the status:

```
NAME          STATUS  AGE
my-feature-store  Ready  3h1m
```

2. Verify that OIDC authorization is configured by checking the status conditions:

```
$ oc get feast <feast_name> -n <namespace> -o jsonpath='{.status.conditions[?(@.type=="Authorization")]}' | python3 -m json.tool
```

If the procedure was successful, the output shows **"type": "Authorization"** with **"status": "True"**.

```
{
  "lastTransitionTime": "2026-04-30T19:57:44Z",
  "message": "OIDC authorization installation complete",
  "reason": "Ready",
  "status": "True",
  "type": "Authorization"
}
```

3. Test your authentication by making requests to the Feature Store REST API.
Get the Feature Store registry REST route:

```
$ FEAST_ROUTE=$(oc get route -n <namespace> -l feast.dev/service-type=registry -o
jsonpath='{.items[0].spec.host}')
```

Obtain a valid OIDC token and store it in the **TOKEN** variable:

```
$ TOKEN=$(oc whoami --token)
```

With the valid OIDC token, verify that the request returns a **200** response:

```
$ curl -sk -w "\nHTTP Status: %{http_code}\n" \
-H "Authorization: Bearer $TOKEN" \
"https://$FEAST_ROUTE/feature_views/all"
```

With an invalid token, verify that the request returns a **401 Unauthorized** response:

```
$ curl -sk -w "\nHTTP Status: %{http_code}\n" \
-H "Authorization: Bearer invalid-token" \
"https://$FEAST_ROUTE/feature_views/all"
```

2.3.3. Accessing Feature Store data by using a single sign-on

You can access Feature Store from the OpenShift AI dashboard or a workbench without providing additional authentication. The system automatically passes your OpenID Connect (OIDC) token or service account token to the Feature Store API.

Prerequisites

- You have configured your cluster and Feature Store instance to use an external OIDC provider.
- You have logged in to the OpenShift AI platform with your OIDC provider credentials.
- Your OIDC token has the required claims, such as membership in the correct groups and namespaces, to pass the Feature Store role-based access control (RBAC) policies.

Procedure

1. Open the OpenShift AI dashboard or a workbench, such as a Jupyter notebook.
2. Query Feature Store for training features.
The system automatically passes your OIDC token or service account token to the Feature Store API.

The following code example shows how to retrieve online features:

```
from feast import FeatureStore

store = FeatureStore(repo_path="/path/to/feature_repo")
features = store.get_online_features(
    features=["my_feature_view:feature1"],
    entity_rows={"entity_id": 1}
).to_dict()
print(features)
```

Verification

- Verify that the query returns the requested features.



NOTE

If the query fails, check the following:

- An **Unauthorized (401)** response indicates that your token has expired or is invalid.
- A **Forbidden (403)** response indicates that your groups or namespaces do not have the required permissions.

CHAPTER 3. DEFINING MACHINE LEARNING FEATURES

As part of the Feature Store workflow, machine learning (ML) engineers or data scientists are responsible for identifying data sources and defining features of interest.

3.1. SETTING UP YOUR WORKING ENVIRONMENT

You must set up your Red Hat OpenShift AI working environment so that you can use features in your machine learning workflow.

Prerequisites

- You have access to the OpenShift AI project in which your cluster administrator has set up the Feature Store instance.

Procedure

1. From the OpenShift AI dashboard, click **Projects**.
2. Click the name of the project in which your cluster administrator has set up the Feature Store instance.
3. In the project in which the cluster administrator set up Feature Store, create a workbench, as described in [Creating a workbench](#).
4. To open the IDE (for example, JupyterLab), in a new window, click the open icon () next to the workbench.
5. Add a **feature_store.yaml** file to your notebook environment. For example, upload a local file or clone a Git repo that contains the file, as described in [Uploading an existing notebook file to JupyterLab from a Git repository by using the CLI](#).
6. Open a new Python notebook.
7. In a cell, run the following command to install the **feast** CLI:

```
! pip install feast
```

Verification

1. Run the following command to list the available features:

```
! feast features list
```

The output should show a list of features, Feature View and data type similar to the following:

```
Feature Feature      View      Data Type
credit_card_due      credit_history Int64
mortgage_due         credit_history Int64
student_loan_due     credit_history Int64
vehicle_loan_due     credit_history Int64
```

```
city          zipcode_features String
state         zipcode_features String
location_type zipcode_features String
```

- Optionally, run the following commands to list the registered feast projects, feature views, and entities.

```
! feast projects list

! feast feature-views list

! feast entities list
```

3.2. ABOUT FEATURE DEFINITIONS

A machine learning feature is a measurable property or *field* within a data set that a machine learning model can analyze to learn patterns and make decisions. In Feature Store, you define a feature by defining the name and data type of a field.

A feature definition is a schema that includes the field name and data type, as shown in the following example:

```
from feast import Field
from feast.types import Int64

credit_card_amount_due = Field(
    name="credit_card_amount_due",
    dtype=Int64
)
```

For a list of supported data types for fields in Feature Store, see the [feast.types module](#) in the Feast documentation.

In addition to field name and data type, a feature definition can include additional metadata, specified as descriptions of features, as shown in the following example:

```
from feast import Field
from feast.types import Int64

credit_card_amount_due = Field(
    name="credit_card_amount_due",
    dtype=Int64,
    description="Credit card amount due for user",
    tags={"team": "loan_department"},
)
```

3.3. UNDERSTANDING FEATURE STORE DATA TYPES

Feature Store supports multiple data types for building features for real-time model serving and maintaining compliance with data handling regulations.

3.3.1. Data type categories

Feature Store organizes data types into six categories:

Primitive types

Basic scalar values such as integers, floats, strings, booleans, timestamps, and UUIDs.

Array types

Ordered lists of values for any primitive type.

JSON types

Opaque JSON data stored as strings at the protocol buffer level, with native JSON support in compatible backends.

Map types

Dictionary-like structures with string keys and values of any supported Feature Store type, including nested maps.

Set types

Collections of unique values for any primitive type. Duplicates are automatically removed.

Struct types

Schema-aware structured types with named, typed fields. Unlike maps, structs declare field names and types for schema validation.

3.3.2. Choosing data types

Consider backend compatibility and performance when selecting data types. Different backends provide varying levels of native support for complex types such as JSON, Map, and Struct. For complete type specifications, backend support tables, and usage examples, see *Feature Store data type reference*.

Additional resources

- [Feast data types](#)

3.4. FEATURE STORE DATA TYPE REFERENCE

You can reference the data type specifications, backend support tables, and usage examples for Feature Store.

3.4.1. Primitive data types

The following table represents all the primitive data types that Feature Store supports:

Feature Store Type	Python Type	Description
Int32	int	32-bit signed integer
Int64	int	64-bit signed integer
Float32	float	32-bit floating point
Float64	float	64-bit floating point
String	str	string/text value

Feature Store Type	Python Type	Description
Bytes	bytes	Binary data
Bool	bool	Boolean value
UnixTimestamp	datetime	Unix timestamp (nullable)
Uuid	uuid.UUID	UUID (any version)
TimeUuid	uuid.UUID	Time-based UUID (version 1)
Decimal	decimal.Decimal	Arbitrary-precision decimal number

3.4.2. Array data types

All primitive types have corresponding array types for storing lists of values. The following table shows the available array data types:

Feature Store type	Python type	Description
Array(Int32)	List[int]	A list of 32-bit integers.
Array(Int64)	List[int]	List of 64-bit integers
Array(Float32)	List[float]	List of 32-bit floats
Array(Float64)	List[float]	List of 64-bit floats
Array(String)	List[str]	List of strings
Array(Bytes)	List[bytes]	List of binary data
Array(Bool)	List[bool]	List of booleans
Array(UnixTimestamp)	List[datetime]	List of timestamps
Array(Uuid)	List[uuid.UUID]	A list of UUIDs.
Array(TimeUuid)	List[uuid.UUID]	List of time-based UUIDs
Array(Decimal)	List[decimal.Decimal]	List of arbitrary-precision decimals

3.4.3. JSON data types

The JSON type represents opaque JSON data. Unlike a Map, which provides schema-free key-value storage, the system stores JSON as a string at the protobuf level. However, backends use native JSON types when available. The following table provides the type, Python type and description:

Feature Store Type	Python Type	Description
JSON	str (JSON-encoded)	JSON data stored as a string in protocol buffers.
Array(Json)	List[str]	A list of JSON-encoded strings.

3.4.3.1. Backend support for JSON data types

The following table shows the native types that each backend uses to support JSON data types:

Backend	Native type
PostgreSQL	jsonb
Snowflake	JSON / VARIANT
Redshift	json
BigQuery	JSON
Spark	Not natively distinguished from String
MSSQL	nvarchar(max)



NOTE

If the native type of a backend is ambiguous, such as when PostgreSQL jsonb could be Map or JSON, the schema-declared Feature Store type takes precedence. Feature Store uses the backend-to-Feast mappings only during schema inference when you do not provide an explicit type.

3.4.4. Map data types

You can store dictionary-like data structures using map data types. The following table shows the available map data types:

Feature Store Type	Python Type	Description
Map	Dict[str, Any]	A dictionary with string keys and values of any supported Feature Store type, including nested maps.

Feature Store Type	Python Type	Description
Array(Map)	List[Dict[str, Any]]	A list of dictionaries.

**NOTE**

You must use strings for map keys, but you can use any supported Feature Store type for map values, such as primitives, arrays, or nested maps.

3.4.4.1. Backend support for map data types

The following table shows the native types that each backend uses to support map data types:

Backend	Native Type	Notes
PostgreSQL	jsonb, jsonb[]	jsonb → Map, jsonb[] → Array(Map)
Snowflake	VARIANT, OBJECT	Inferred as Map
Redshift	SUPER	Inferred as Map
Spark	map<string,string>	map<> → Map, array<map<>> → Array(Map)
Athena	map	Inferred as Map
MSSQL	nvarchar(max)	Serialized as string
DynamoDB / Redis	Proto bytes	Full proto Map support

3.4.4.2. Map type usage examples

You can use maps to store complex nested data structures. The following table shows several map options:

```
# Simple map
user_preferences = {
    "theme": "dark",
    "language": "en",
    "notifications_enabled": True,
    "font_size": 14
}

# Nested map
metadata = {
    "profile": {
        "bio": "Software engineer",
        "location": "San Francisco"
    }
}
```

```
    },
    "stats": {
      "followers": 1000,
      "posts": 250
    }
  }

# List of maps
activity_log = [
  {"action": "login", "timestamp": "2024-01-01T10:00:00", "ip": "192.168.1.1"},
  {"action": "purchase", "timestamp": "2024-01-01T11:30:00", "amount": 99.99},
  {"action": "logout", "timestamp": "2024-01-01T12:00:00"}
]
```

3.4.5. Set data types

All primitive types have corresponding set types to store unique values. The following table shows the available set data types:

Feature Store type	Python type	Description
Set(Int32)	Set[int]	Set of unique 32-bit integers
Set(Int64)	Set[int]	Set of unique 64-bit integers
Set(Float32)	Set[float]	Set of unique 32-bit floats
Set(Float64)	Set[float]	Set of unique 64-bit floats
Set(String)	Set[str]	Set of unique strings
Set(Bytes)	Set[bytes]	Set of unique binary data
Set(Bool)	Set[bool]	Set of unique booleans
Set(UnixTimestamp)	Set[datetime]	Set of unique timestamps
Set(Uuid)	Set[uuid.UUID]	Set of unique UUIDs
Set(TimeUuid)	Set[uuid.UUID]	Set of unique time-based UUIDs
Set(Decimal)	Set[decimal.Decimal]	Set of unique arbitrary-precision decimals



NOTE

When you convert lists or other iterables to sets, the set automatically removes duplicate values.

3.4.6. Struct data types

The Struct type provides a data structure with named, typed fields. You must declare field names and their types to enable schema validation. The following table shows the available struct data types:

Feature Store Type	Python Type	Description
Struct({"field": Type, ...})	Dict[str, Any]	This includes named fields with typed values.
Array(Struct({"field": Type, ...}))	List[Dict[str, Any]]	A list of structs.

Struct data types example

```
from feast.types import Struct, String, Int32, Array

# Struct with named, typed fields
address_type = Struct({"street": String, "city": String, "zip": Int32})
Field(name="address", dtype=address_type)

# Array of structs
items_type = Array(Struct({"name": String, "quantity": Int32}))
Field(name="order_items", dtype=items_type)
```

3.4.6.1. Backend support for struct data types

The following table shows the native types that each backend uses to support struct data types:

Backend	Native Type
BigQuery	STRUCT / RECORD
Spark	struct<...> / array<struct<...>>
PostgreSQL	jsonb (serialized)
Snowflake	VARIANT (serialized)
MSSQL	nvarchar(max) (serialized)
DynamoDB / Redis	Proto bytes

3.4.7. Complete feature view example

The following example demonstrates a feature view that uses multiple data types:

```
from datetime import timedelta
from feast import Entity, FeatureView, Field, FileSource
from feast.types import (
```

```

Int32, Int64, Float32, Float64, String, Bytes, Bool, UnixTimestamp,
Uuid, TimeUuid, Decimal, Json, Array, Set, Map, Struct
)

# Define a data source
user_features_source = FileSource(
    path="data/user_features.parquet",
    timestamp_field="event_timestamp",
)

# Define an entity
user = Entity(
    name="user_id",
    description="User identifier",
)

# Define a feature view with all supported types
user_features = FeatureView(
    name="user_features",
    entities=[user],
    ttl=timedelta(days=1),
    schema=[
        # Primitive types
        Field(name="age", dtype=Int32),
        Field(name="account_balance", dtype=Int64),
        Field(name="transaction_amount", dtype=Float32),
        Field(name="credit_score", dtype=Float64),
        Field(name="username", dtype=String),
        Field(name="profile_picture", dtype=Bytes),
        Field(name="is_active", dtype=Bool),
        Field(name="last_login", dtype=UnixTimestamp),
        Field(name="session_id", dtype=Uuid),
        Field(name="event_id", dtype=TimeUuid),
        Field(name="price", dtype=Decimal),

        # Array types
        Field(name="daily_steps", dtype=Array(Int32)),
        Field(name="transaction_history", dtype=Array(Int64)),
        Field(name="ratings", dtype=Array(Float32)),
        Field(name="portfolio_values", dtype=Array(Float64)),
        Field(name="favorite_items", dtype=Array(String)),
        Field(name="document_hashes", dtype=Array(Bytes)),
        Field(name="notification_settings", dtype=Array(Bool)),
        Field(name="login_timestamps", dtype=Array(UnixTimestamp)),
        Field(name="related_session_ids", dtype=Array(Uuid)),
        Field(name="event_chain", dtype=Array(TimeUuid)),
        Field(name="historical_prices", dtype=Array(Decimal)),

        # Set types (unique values only)
        Field(name="visited_pages", dtype=Set(String)),
        Field(name="unique_categories", dtype=Set(Int32)),
        Field(name="tag_ids", dtype=Set(Int64)),
        Field(name="preferred_languages", dtype=Set(String)),
        Field(name="unique_device_ids", dtype=Set(Uuid)),
        Field(name="unique_event_ids", dtype=Set(TimeUuid)),
        Field(name="unique_prices", dtype=Set(Decimal)),
    ]
)

```

```

# Map types
Field(name="user_preferences", dtype=Map),
Field(name="metadata", dtype=Map),
Field(name="activity_log", dtype=Array(Map)),

# Nested collection types
Field(name="weekly_scores", dtype=Array(Array(Float64))),
Field(name="unique_tags_per_category", dtype=Array(Set(String))),

# JSON type
Field(name="raw_event", dtype=Json),

# Struct type
Field(name="address", dtype=Struct({"street": String, "city": String, "zip": Int32})),
Field(name="order_items", dtype=Array(Struct({"name": String, "qty": Int32}))),
],
source=user_features_source,
)

```

3.5. SPECIFYING THE DATA SOURCE FOR FEATURES

As an ML engineer or a data scientist, you must specify the data source for the features that you want to define.

The data source differs depending on whether you are using an offline store, for batch data and training data sets, or an online store, for model inference. Optionally, you can use a Parquet or a Delta-formatted file as the data source. You can specify a local file or a file in storage, such as Amazon Simple Storage Service (S3).

For offline stores, specify a batch data source. You can specify a data warehouse, such as BigQuery, Snowflake, Redshift, or a data lake, such as Amazon S3 or Google Cloud Platform (GCP). You can use Feature Store to ingest and query data across both types of data sources.

For online stores, specify a database backend, such as Redis, GCP Datastore, or DynamoDB.

Prerequisites

- You know the location of the data source for your ML workflow.

Procedure

1. In the editor of your choice, create a new Python file.
2. At the beginning of the file, specify the data source for the features that you want to define within the file.

For example, use the following code to specify the data source as a Parquet-formatted file:

```

from feast import FileSource
from feast.data_format import ParquetFormat

parquet_file_source = FileSource(
    file_format=ParquetFormat(),
    path="file:///feast/customer.parquet",
)

```

3. Save the file.

3.6. ABOUT ORGANIZING FEATURES BY USING ENTITIES

Within a feature view, you can group features that share a conceptual link or relationship together to define an entity. You can think of an entity as a primary key that you can use to fetch features. Typically, an entity maps to the domain of your use case. For example, a fraud detection use case could have customers and transactions as their entities, with group-related features that correspond to these customers and transactions.

A feature does not have to be associated with an entity. For example, a feature of a customer entity could be the number of transactions they have made on an average month, while a feature that is not observed on a specific entity could be the total number of transactions made by all users in the last month.

```
customer = Entity(name='dob_ssn', join_keys=['dob_ssn'])
```

The entity name uniquely identifies the entity. The join key identifies the physical primary key on which feature values are joined together for feature retrieval.

The following table shows example data with a single entity column (**dob_ssn**) and two feature columns (**credit_card_due** and **bankruptcies**).

Table 3.1. Example data showing an entity and features

row	timestamp	dob_ssn	credit_card_due	bankruptcies
1	5/22/2025 0:00:00	19530219_5179	833	0
2	5/22/2025 0:00:00	19500806_6783	1297	0
3	5/22/2025 0:00:00	19690214_3370	3912	1
4	5/22/2025 0:00:00	19570513_7405	8840	0

3.7. CREATING FEATURE VIEWS

You define features within a feature view. A *feature view* is an object that represents a logical group of time-series feature data in a data source. Feature views indicate to Feature Store where to find your feature values, for example, in a parquet file or a BigQuery table.

By using feature views, you define the existing feature data in a consistent way for both an offline environment, when you train your models, and an online environment, when you want to serve features to models in production.

Feature Store uses feature views during the following tasks:

- Generating training datasets by querying the data source of feature views to find historical feature values. A single training data set can consist of features from multiple feature views.
- Loading feature values into an online or offline store. Feature views determine the storage schema in the online or offline store. Feature values can be loaded from batch sources or from stream sources.

- Retrieving features from the online or offline store. Feature views provide the schema definition for looking up features from the online or offline store.

When you create a feature project, the **feature_repo** subfolder includes a Python file that includes example feature definitions (for example, **example_features.py**).

To define new features, you can edit the code in the example file or add a new file to the feature repository.

Note: Feature views only work with timestamped data. If your data does not contain timestamps, insert dummy timestamps. The following example shows how to create a table with dummy timestamps for PostgreSQL-based data:

```
CREATE TABLE employee_metadata (
  employee_id INT PRIMARY KEY,
  department TEXT,
  dummy_event_timestamp TIMESTAMP DEFAULT '2024-01-01'
);
INSERT INTO employee_metadata (employee_id, department)
VALUES (1, 'Advanced'), (2, 'New');
```

Prerequisites

- You know what data is relevant to your use case.
- You have identified attributes in your data that you want to use as features in your ML models.

Procedure

1. In your IDE, such as JupyterLab, open the **feature_repo/example_features.py** file that contains example feature definitions or create a new Python (.py) file in the **feature_repo** directory.
2. Create a feature view that is relevant to your use case based on the structure shown in the following example:

```
credit_history_source = FileSource( ❶
  name="Credit history",
  path="data/credit_history.parquet",
  file_format=ParquetFormat(),
  timestamp_field="event_timestamp",
  created_timestamp_column="created_timestamp",
)
credit_history = FeatureView( ❷
  name="credit_history",
  entities=[dob_ssn], ❸
  ttl=timedelta(days=90), ❹
  schema=[ ❺
    Field(name="credit_card_due", dtype=Int64),
    Field(name="mortgage_due", dtype=Int64),
    Field(name="student_loan_due", dtype=Int64),
    Field(name="vehicle_loan_due", dtype=Int64),
    Field(name="hard_pulls", dtype=Int64),
    Field(name="missed_payments_2y", dtype=Int64),
```

```

    Field(name="missed_payments_1y", dtype=Int64),
    Field(name="missed_payments_6m", dtype=Int64),
    Field(name="bankruptcies", dtype=Int64),
],
source=credit_history_source, 6
tags={"origin": "internet"}, 7
)

```

- 1** A data source that provides time-stamped tabular data. A feature view must always have a data source for the generation of training datasets and when materializing feature values into the online store. Possible data sources are batch data sources from data warehouses (BigQuery, Snowflake, Redshift), data lakes (S3, GCS), or stream sources. Users can push features from data sources into Feature Store, and make the features available for training or batch scoring ("offline"), for realtime feature serving ("online"), or both.
- 2** A name that identifies the feature view in the project. Within a feature view, feature names must be unique.
- 3** Zero or more entities. Feature views generally contain features that are properties of a specific object, in which case that object is defined as an entity and included in the feature view. If the features are not related to a specific object, the feature view might not have entities.
- 4** (Optional) Time-to-live (TTL) to limit how far back to look when Feature Store generates historical datasets.
- 5** One or more feature definitions.
- 6** A reference to the data source.
- 7** (Optional) You can add metadata, such as tags that enable filtering of features when viewing them in the UI, listing them by using a CLI command, or by querying the registry directly.

3. Save the file.

CHAPTER 4. RETRIEVING FEATURES FOR MODEL TRAINING

4.1. RETRIEVING DATA SCIENCE FEATURES

You can connect to the Feature Store and consume the features necessary for model development and inference.

Prerequisites

- Your Feature Store has been deployed.
- User access permissions have been configured by the administrator.
- You have access to a relevant project and a workbench.
- Your Feature Store client configuration must be complete.

Procedure

1. From the OpenShift AI dashboard, click **Projects**.
The **Projects** page opens.
2. Click the name of the project that you want to work on.
3. Create a new workbench or open an existing workbench with the Feast software development kit (SDK).
 - a. In the Feature Store client configuration table, select the configmaps associated with the desired repositories.
 - b. Copy the Python script that is generated on the left side of the page. that is generated on the left of the page.
 - c. Click the **Workbenches** tab and launch a workbench.
 - d. Paste the Python script into the workbench cell.



NOTE

- You do not need tokens or manual authentication for the Feature Store. Your workbench automatically authenticates using the access your administrator granted to your Data Science Project.
- Feast SDK is available in all images except minimal.

CHAPTER 5. FEATURE STORE INTEGRATION WITH WORKBENCHES

The integration of Feature Store with Red Hat workbenches resolves the competing demands of agility and security. This capability removes the friction of manual configuration and script copying, accelerating workflows and enabling seamless feature management directly within Jupyter notebooks. A governance framework ensures efficiency, scalable, role-based access control (RBAC), and secure authentication. This safeguards sensitive data and mitigates vulnerabilities enterprise-wide.

5.1. CONNECTING A WORKBENCH TO A FEATURE STORE

You can connect your workbench to a Feature Store instance to avoid manual configuration and immediately access features for your models.

Prerequisites

- You have logged in to the Red Hat dashboard.
- You have the necessary system permissions to view the specific Feature Store instances you intend to connect.
- A Feature Store instance is deployed and available in your OpenShift cluster.

You can avoid manual scripting by selecting a preconfigured Feature Store during the workbench provisioning process.

Procedure

- Log in to the Red Hat Red Hat dashboard, and click **Data Science Projects**.
- Click the name of the project that contains the workbench you want to configure.
- Perform one of the following actions:
 - Create a new workbench: In the workbenches section, click **Create workbench**.
 - Edit an existing workbench: In the workbenches section, click the **More options** icon (**:**) beside the workbench name and click **Edit workbench**.
- Scroll to the Feature Store Configuration section.
- Select one or more available Feature Store instances from the **Select Feature Store** list.
- Follow the instructions in the **Example code** dialog box under the **Feature Store selection** dropdown menu.



NOTE

The **Select Feature Store** list displays only the Feature Store instances that your user account is authorized to access.

- Click **Create workbench** or **Update workbench**. The workbench starts with the Feature Store configuration files automatically mounted, enabling you to efficiently integrate features into your notebooks.

Verification

Verify the success of the procedure by creating a Python object that uses the Feature Store client configuration. Copy and paste the following code:

```
fs_banking = FeatureStore(fs_yaml_file=' /opt/app-root/config/feast_configs/fs_banking.yaml')
fs_banking.list_feature_views()
fs_banking.get_historical_features(. . . .)
fs_banking.get_online_features(. . . .)
```

Additional resources

- [Jupyter Notebooks](#)

5.2. VERIFYING ACCESS CONTROL TO YOUR FEATURE STORE INSTANCES

You must ensure that access to Feature Stores is restricted based on defined roles and permissions.

To maintain regulatory compliance and safeguard sensitive data, the system uses role-based access control (RBAC) to determine which Feature Stores are visible to specific users. This mechanism mitigates potential vulnerabilities by preventing unauthorized users from discovering or selecting restricted Feature Stores.

Prerequisites

- You must be logged into the Red Hat dashboard.
- You must have the **feast-configs-registry** ConfigMap present in your infrastructure namespace (e.g., `opendatahub` or `redhat-ods-applications`).

Procedure

- Define the specific user groups, namespaces or data science projects, requiring access to a Feature Store instance in your **Data science projects** permissions tab.
- Assign the appropriate RBAC permissions to user groups or data science projects in the Feature Store configuration maps.

Verification

Verify the configuration by logging in as a user with restricted permissions to a specific feature store:

- Attempt to create a workbench.
- Confirm that the **Select Feature Store** list displays only the instances the user is authorized to access.
- You can also log in as a user with no permissions and confirm that no Feature Stores are selectable.

Your data access is now protected, ensuring that different personas access only the features appropriate for their role.

Additional resources

For more information about RBAC, see the following: [Setting Up Kubernetes RBAC for Feast](#)



NOTE

The RBAC document displays all policies available in the Feast upstream. However, Red Hat supports Group and Namespace/Data_Science_Project based authorization policies only. Role-based policies are *NOT* supported.

5.3. FEATURE STORE CONFIGURATION REFERENCE

Understand how the system mounts configuration files to enable you to create and use features in your code.

Configuration element	Description
Configuration path	The system mounts the necessary Feature Store client configuration files to the following Feast configuration file: /opt/app-root/src/feast-config/my_project.yaml
Python instantiation	Initialize Feature Store with the path to the mounted configuration file: fs_obj = FeatureStore(fs_yaml_file='/usr/app-root/config/feast/my_project.yaml')
Troubleshooting access	If you cannot view a specific Feature Store in the workbench dropdown menu, an administrator must verify a couple items for you. They need to verify that you have the correct role-based access control (RBAC) based on your group/namespace access and that your authentication token is valid and permits viewing a specific config map associated with that Feature Store instance.

CHAPTER 6. UNDERSTANDING COMPUTE ENGINES IN FEATURE STORE

You can configure compute engines to manage feature pipelines, transformations, and materialization in Red Hat. By integrating your Feature Store with distributed processing frameworks, you can centralize feature management and improve data reusability across your organization.

6.1. USING COMPUTE ENGINES IN FEATURE STORE

You can use compute engines to run feature pipelines on back ends such as Spark, PyArrow, Pandas, or Ray. These pipelines perform transformations, aggregations, joins, and materializations.

Use the compute engine to build and run directed acyclic graphs (DAGs), for modular and scalable workflows.

Available operations:

- **materialize()**: Generate features for offline and online stores in batch and stream modes.
- **get_historical_features()**: Retrieve point-in-time training datasets.

Key concepts for compute engines

Understand the following components for better execution of materialization and retrieval tasks:

Concept	Definition
Compute engine	The interface for executing materialization and retrieval tasks.
Feature builder	Constructs a Directed Acyclic Graphs (DAG), from a feature view definition for a specific backend.
Feature resolver	Arranges tasks in the correct sequence, so each step runs only after its dependencies.
DAG	A DAG operation, such as read, aggregate, or join.
Execution plan	Runs nodes in the correct sequence and saves the results.
Execution Context	Collects configuration, registry, stores, entity data, and node outputs.

Understanding the feature resolver and builder

The feature builder starts a feature resolver that extracts a DAG from **FeatureView** definitions, resolving dependencies and ensuring the correct execution order. A **FeatureView** represents a logical data source, whereas a **DataSource** represents the physical data source.

When defining a feature view, the source can be a physical data source, a derived feature view, or a list

of feature views. Use the feature resolver to organize data sources into a directed acyclic graph (DAG). The resolver identifies node dependencies to generate the final output. The FeatureBuilder then builds DAG node objects for each operation, such as read, join, filter, or aggregate.

Table 6.1. Supported Compute engines

Compute engine	Description
Spark compute engine	Distributed DAG execution using Apache Spark. Supports point-in-time joins and large-scale materialization. Integrates with Spark Offline Store and Spark materialization job.
Ray compute engine	Provides distributed DAG execution. Enables automatic resource management and optimization. Integrates with Ray Offline Store and Ray Materialization Job.
Local compute engine	Runs on Arrow and a backend you specify (e.g., Pandas, Polars).
Enables local development, testing, or lightweight feature generation.	Supports local materialization job and local historical retrieval job .

Feature builder node details

Use the feature builder to build a directed acyclic graph (DAG) from a feature view definition to determine the operation order. The feature resolver identifies data sources and sorts the nodes to resolve dependencies.

Table 6.2. Feature builder nodes

Node type	Description
Source read node	The process begins by reading the data source.
Transformation node or join node	If a feature transformation is defined, the system applies a transformation node. If there are multiple sources the system applies a join node.
Filter node	The system always includes this node to apply time to live (TTL) parameters or user-defined filters.
Aggregation node	The system applies this node if the feature view includes defined aggregations.
Deduplication node	The system applies this node for get_historical_features requests if no aggregation is defined.

Node type	Description
Validation node	The system applies this node if enable_validation is set to true.
Output	Use retrieval output for get_historical_features requests. Use online store write or offline store write, for materialize requests.

6.2. UNDERSTANDING THE RAY COMPUTE ENGINE IN FEATURE STORE

The Ray compute engine is a distributed compute implementation that uses Ray for executing feature pipelines. This includes transformations, aggregations, joins, and materializations. It provides scalable and efficient distributed processing for both **materialize()** and **get_historical_features()** operations.

The Ray RAG template features:

- **Parallel embedding generation:** Uses the Ray compute engine to generate embeddings across multiple workers
- **Vector search integration:** Works with Milvus for semantic similarity search
- **Complete RAG pipeline:** The Ray compute engine automatically distributes the embedding generation across available workers, making it ideal for processing large datasets efficiently

Ray compute engine features:

- **Distributed directed acyclic graphs (DAG) Execution:** Executes feature computation DAG across Ray clusters
- **Intelligent Join Strategies:** Automatic selection between broadcast and distributed joins
- **Lazy Evaluation:** Deferred execution for optimal performance
- **Resource Management:** Automatic scaling and resource optimization
- **Point-in-Time Joins:** Efficient temporal joins for historical feature retrieval

6.3. GETTING STARTED USING THE RAY TEMPLATE

Use the Ray retrieval-augmented generation (RAG) template to build scalable, high-performance applications. This end-to-end framework enables parallel processing of large datasets, which reduces the processing time and memory intensity required on a single machine.

Prerequisites

- You have a data science project with an active workbench.
- Your workbench image includes the Feature Store.

Procedure

1. Apply the Ray RAG Template
- Run the following code for RAG (Retrieval-Augmented Generation) applications with distributed embedding generation: **feast init -t ray_rag my_rag_project cd my_rag_project/feature_repo**

Your Ray template is now active.

6.4. CONFIGURING RAY IN YOUR FEATURE STORE YAML FILE

Configure the Ray compute engine in Feature Store by defining Ray-specific settings in the **feature_store.yaml** file. This enables distributed execution of feature pipelines for materialization and historical feature retrieval.

Prerequisites

- Your Ray cluster is running.

Procedure

1. Configure the Ray compute engine in your **feature_store.yaml** file:

YAML	Available options
project: my_project	None
registry: data/registry.db	None
provider: local	None
offline_store: type: ray storage_path: data/ray_storage	None
batch_engine: type: ray.engine max_workers: 4	Maximum number of workers
broadcast_join_threshold_mb: 100	Broadcast join threshold (MB)
max_parallelism_multiplier: 2	Parallelism multiplier

<code>target_partition_size_mb: 64</code>	Target partition size (MB)
<code>window_size_for_joins: "1H"</code>	Time window for distributed joins
<code>ray_address: localhost</code>	Ray cluster address

Table 6.3. Ray Configuration options in Feature Store

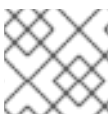
Option	Type	Default	Description
<code>type</code>	string	ray.engine	Must be ray.engine
max_workers	integer	none (uses all cores)	This enables the maximum number of Ray workers.
enable_optimization	boolean	true	This enables performance optimizations.
broadcast_join_threshold_mb	integer	100	This is the size threshold for broadcast joins (MB).
max_parallelism_multiplier	integer	2	This enables you to run many CPU cores simultaneously.
target_partition_size_mb	integer	64	This allows you to identify a partition size (MB).
window_size_for_joins	string	1H	This enables a time window for distributed joins.
ray_address	string	none	This enables the Ray cluster address, which triggers the remote mode.
use_kuberay	boolean	none	This enables KubeRay mode (overrides ray_address).

Option	Type	Default	Description
kuberay_conf	dictionary	none	This enables KubeRay configuration dictionary with keys: cluster_name (required), namespace (default: default), auth_token , auth_server , skip_tls (default: false).
enable_ray_logging	boolean	false	This enables Ray progress bars and logging.
enable_distributed_joins	boolean	true	This enables distributed joins for large datasets.
staging_location	string	none	This is the remote path for batch materialization jobs.
ray_conf	dictionary	none	These are Ray configuration parameters such as memory and CPU limits.

6.5. UNDERSTANDING RAY MODE DETECTION PRECEDENCE IN FEATURE STORE

You can manage mode detection precedence since the Ray compute engine automatically detects the execution mode:

- Environment Variables (→) KubeRay mode (if **FEAST_RAY_USE_KUBERAY=true**)
- Config **kuberay_conf** (→) KubeRay mode
- Config **ray_address** (→) Remote mode
- Default (→) Local mode



NOTE

It is recommended that you use KubeRay mode.



NOTE

For more information about Ray compute engine usage examples, see Ray compute engine usage examples.

6.6. USING RAY DIRECTED ACYCLIC GRAPH NODE TYPES IN FEATURE STORE

Use Ray directed acyclic graph (DAG) node types to build scalable, high-performance feature generation workflows. Ray optimizes resources and reduces execution time by handling data partitioning and statically allocating buffers.

Ray read node

- a. Reads data from Ray-compatible sources:
 - Supports Parquet, comma-separated values (CSV), and other formats
 - Handles partitioning and schema inference
 - Applies field mappings and filters

Ray join node

- a. Performs distributed joins:
 - **Broadcast join:** Use for small datasets (<100MB)
 - **Distributed join:** Use for large datasets with time-based windowing
 - **Automatic strategy selection:** Based on dataset size and cluster resources

Ray filter node

- a. Applies filters and time-based constraints:
 - Time to live (TTL)-based filtering
 - Timestamp range filtering
 - Custom predicate filtering

Ray aggregation node

- a. Handles feature aggregations:
 - Windowed aggregations
 - Grouped aggregations
 - Custom aggregation functions

Ray transformation node

- a. Applies feature transformations:
 - Row-level transformations
 - Column-level transformations
 - Custom transformation functions

Ray write node

- a. Writes results to various targets:
 - Online stores
 - Offline stores
 - Temporary storage

6.7. USING RAY JOIN STRATEGIES IN FEATURE STORE

The Ray compute engine automatically selects optimal join strategies:

.Used for small feature datasets:

- Selects this join automatically when feature data is <100MB.
- Caches features in Ray's object store.
- Distributes entities across a cluster.
- Copies feature data and sends it to each worker.
- Uses the distributed window join.
- Used for large feature datasets.
- Selects this join automatically when feature data <100MB.
- Partitions data by time windows.
- Provides point-in-time joins within each window.
- Combines results across windows.

Example of using strategy selection logic

```
def select_join_strategy(feature_size_mb, threshold_mb):  
    if feature_size_mb < threshold_mb:  
        return "broadcast"  
    else:  
        return "distributed_windowed"
```

6.8. UNDERSTANDING RAY PERFORMANCE OPTIMIZATION FOR FEATURE STORE

Ray is a distributed execution engine that scales Feast feature engineering and retrieval-augmented generation (RAG) workloads. By processing large datasets in parallel, Ray accelerates pipelines and reduces costs compared to single-node processing.

Use the Ray automatic optimizations for increased efficiency.

1. Enabling automatic optimization

- a. The Ray compute engine includes several automatic optimizations:
- **Partition optimization:** Automatically determines optimal partition sizes
 - **Join strategy selection:** Chooses between broadcast and distributed joins
 - **Resource allocation:** Scales workers based on available resources
 - **Memory management:** Handles out-of-core processing for large datasets

Manual tuning example

If you have specific workloads that require custom tuning, you can fine-tune performance:

```
batch_engine:
  type: ray.engine
  # Fine-tuning for high-throughput scenarios
  broadcast_join_threshold_mb: 200    # Larger broadcast threshold
  max_parallelism_multiplier: 1      # Conservative parallelism
  target_partition_size_mb: 512      # Larger partitions
  window_size_for_joins: "2H"       # Larger time windows
```

6.8.1. Understanding Ray monitoring and metrics in Feature Store

You can check cluster resources and monitor job progress when working with the Ray compute engine.

See the following Python example for how to extract monitoring and metrics data:

```
import ray

# Check cluster resources
resources = ray.cluster_resources()
print(f"Available CPUs: {resources.get('CPU', 0)}")
print(f"Available memory: {resources.get('memory', 0) / 1e9:.2f} GB")

# Monitor job progress
job = store.get_historical_features(...)
# Ray compute engine provides built-in progress tracking
```

6.9. UNDERSTANDING THE SPARK COMPUTE ENGINE IN FEATURE STORE

Use the Spark compute engine to run distributed batch materialization and historical retrieval operations. Batch materialization includes materialize and materialize-incremental operations. The engine processes large-scale data from offline stores, such as Snowflake, Google BigQuery, and Apache Spark SQL.

The Spark compute engine can read various data sources and perform distributed or custom transformations. You can use the engine to perform these tasks:

- * Read from various data sources, such as Apache Spark SQL, Google BigQuery, and Snowflake.
- * Execute distributed feature transformations and aggregations.
- * Run custom transformations by using Apache Spark SQL or user-defined functions (UDFs).

6.10. CONFIGURING SPARK IN YOUR FEATURE STORE YAML FILE

Configure the Spark compute engine in Feature Store by defining Spark-specific settings in the `feature_store.yaml` file or programmatically using a Feast RepoConfig. This enables distributed batch materialization and historical feature retrieval using Spark.

Prerequisites

- Your Spark cluster is running.

Procedure

1. Configure the Spark compute engine in your **`feature_store.yaml`** file:

```
...
offline_store:
  type: snowflake.offline
...
batch_engine:
  type: spark.engine
  partitions: 10 # number of partitions when writing to the online or offline store
  spark_conf:
    spark.master: "local[*]"
    spark.app.name: "Feast Spark Engine"
    spark.sql.shuffle.partitions: 100
    spark.executor.memory: "4g"
```

Configuring the Spark offline store example

You can configure the feature store by using the **`feature_store.py`** file. This configuration uses Amazon DynamoDB for the online store and the Spark compute engine for the offline store.



NOTE

In the following code, replace `[YOUR_BUCKET]` with the name of your specific S3 bucket.

```
from feast import FeatureStore, RepoConfig
from feast.repo_config import RegistryConfig
from feast.infra.online_stores.dynamodb import DynamoDBOnlineStoreConfig
from feast.infra.offline_stores.contrib.spark_offline_store.spark import SparkOfflineStoreConfig

repo_config = RepoConfig(
    registry="s3://[YOUR_BUCKET]/feast-registry.db",
    project="feast_repo",
    provider="aws",
    offline_store=SparkOfflineStoreConfig(
        spark_conf={
            "spark.ui.enabled": "false",
            "spark.eventLog.enabled": "false",
            "spark.sql.catalogImplementation": "hive",
            "spark.sql.parser.quotedRegexColumnNames": "true",
            "spark.sql.session.timeZone": "UTC"
        }
    ),
    batch_engine={
        "type": "spark.engine",
```

```

        "partitions": 10
    },
    online_store=DynamoDBOnlineStoreConfig(region="us-west-1"),
    entity_key_serialization_version=3
)
store = FeatureStore(config=repo_config)

```

6.11. REFERENCE MATERIAL FOR INTEGRATING RAY WITH OTHER COMPONENTS IN FEATURE STORE

You can integrate Ray with Spark, cloud storage and feature transformations. This enables distributed processing of large-scale machine learning workloads, from feature engineering to serving. It also enables efficient handling of intensive tasks.

Integrating Ray with the Spark offline store in Feature Store

```

# Use Ray compute engine with Spark offline store
offline_store:
    type: spark
    spark_conf:
        spark.executor.memory: "4g"
        spark.executor.cores: "2"
batch_engine:
    type: ray.engine
    max_workers: 8
    enable_optimization: true

```

Integrating Ray with cloud storage in Feature Store

```

# Use Ray compute engine with cloud storage
offline_store:
    type: ray
    storage_path: s3://my-bucket/feast-data
batch_engine:
    type: ray.engine
    ray_address: "ray://ray-cluster:10001"
    broadcast_join_threshold_mb: 50

```

Integrating Ray with feature transformations

```

from feast import FeatureView, Field
from feast.types import Float64
from feast.on_demand_feature_view import on_demand_feature_view

@on_demand_feature_view(
    sources=["driver_stats"],
    schema=[Field(name="trips_per_hour", dtype=Float64)]
)
def trips_per_hour(features_df):
    features_df["trips_per_hour"] = features_df["avg_daily_trips"] / 24
    return features_df

# Ray compute engine handles transformations efficiently
features = store.get_historical_features(

```

```
entity_df=entity_df,  
features=["trips_per_hour:trips_per_hour"]  
)
```

a. Ray native transformations

If you have distributed transformations that use Ray's dataset and parallel processing capabilities, use **mode=ray** in your **BatchFeatureView**:

```
# Feature view with Ray transformation mode  
document_embeddings_view = BatchFeatureView(  
    name="document_embeddings",  
    entities=[document],  
    mode="ray", # Enable Ray native transformation  
    ttl=timedelta(days=365),  
    schema=[  
        Field(name="document_id", dtype=String),  
        Field(name="embedding", dtype=Array(Float32), vector_index=True),  
        Field(name="movie_name", dtype=String),  
        Field(name="movie_director", dtype=String),  
    ],  
    source=movies_source,  
    udf=generate_embeddings_ray_native,  
    online=True,  
)
```


CHAPTER 7. FEATURE STORE COMMAND LINE INTERFACE REFERENCE

You can use the Feature Store command-line interface (CLI) to manage your Feature Store deployments and repositories. The CLI tool, **feast**, is bundled with the Feature Store Python package and is available immediately after installation. You can run the commands in your workbench.

General usage of command line options

```
feast [OPTIONS] COMMAND [ARGS]...
```

Options

-c, --chdir TEXT

Switch to a different feature repository directory before executing the given subcommand.

--help

Show this message and exit.

7.1. FEATURE STORE GLOBAL COMMAND

You can use the following global options with the **feast** command in your Feature Store workbench:

Table 7.1. Feature Store CLI global options

Option	Description
chdir (-c, --chdir)	Use this global, top-level option with other commands.
feast -c path/to/my/feature/repo apply	Run feast CLI commands in a directory different from the current working directory.

7.2. FEATURE STORE COMMAND LINE INTERFACE OPTIONS

The following table lists the available Feature Store CLI commands. Run these in your workbench.

Table 7.2. Feature Store CLI commands

Command	Description
apply	Create or update a Feature Store deployment.
configuration	Display the Feature Store configuration.
delete	Delete a Feature Store object from the registry.
entities	Access entities.

Command	Description
feature-views	Access feature views.
init	Create a new Feature Store repository.
materialize	Run a non-incremental materialization job to ingest feature data.
materialize-incremental	Run an incremental materialization job to ingest feature data.
registry-dump	Print the contents of the metadata registry.
teardown	Tear down the deployed Feature Store infrastructure.
version	Display the Feature Store SDK version.

7.3. FEATURE STORE APPLY COMMAND

The **feast apply** command updates a Feature Store deployment to match the feature definitions in the feature repository.

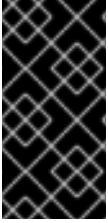
The command performs the following actions:

- **Scans definitions**
Scans the Python files in the feature repository to identify Feature Store object definitions. This includes feature views, entities, and data sources.
- **Validates definitions**
Validates feature definitions to ensure accuracy.
- **Synchronizes metadata**
Synchronizes the metadata of Feature Store objects in the registry. If a registry does not exist, the Feature Store creates one. The standard registry is a **protobuf** binary file stored on a disk, either locally or in an object store.
- **Provisions infrastructure**
Creates the necessary Feature Store infrastructure. The deployed infrastructure depends on the provider configuration specified in the **feature_store.yaml** file:
 - **Local provider:** Creates a SQLite online store.
 - **Cloud provider:** Creates cloud infrastructure for services such as Google Cloud Platform (GCP) or Amazon Web Services (AWS).



NOTE

Creating cloud infrastructure might incur costs.

**IMPORTANT**

The **feast apply** command registers or updates only objects found in your Python files. It does not delete objects that you remove from your code. To delete objects from the registry, use the **feast delete** command or the explicit delete methods available in the Python SDK.

7.4. FEATURE STORE CONFIGURATION COMMAND

The Feature Store configuration command displays the active configuration for the Feature Store environment. The output includes both user-provided and default configurations.

Configuration example command and output

```
Feature Store configuration
project: foo
registry: data/registry.db
provider: local
online_store:
  type: sqlite
  path: data/online_store.db
offline_store:
  type: dask
entity_key_serialization_version: 3
auth:
  type: no_auth
```

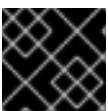
7.5. FEATURE STORE DELETE COMMAND

The **feast delete** command removes a Feast object from the registry. This includes objects such as feature views, entities, data sources, and feature services.

The command searches for the specified object name across all object types, including entities, feature views, feature services, data sources, saved datasets, and validation references. It deletes the first matching object found and removes any associated infrastructure.

Delete syntax

```
feast delete <object_name>
```

**IMPORTANT**

The delete operation is permanent. Proceed with caution.

**NOTE**

If multiple objects share the same name across different types, **feast delete** removes the first one it encounters. For programmatic deletion with greater control, use the Python SDK methods, such as **store.delete_feature_view()** or **store.delete_feature_service()**.

Delete command examples

- Delete a feature view named **driver_hourly_stats**:

```
feast delete driver_hourly_stats
```

- Delete an entity named **driver**:

```
feast delete driver
```

7.6. FEATURE STORE ENTITIES LIST COMMAND

The **feast entities list** command displays a list of all registered entities.

Entities syntax

```
feast entities list [options]
```

Options

--tags <text>

Filters the list by tags (for example, **--tags 'key:value'**). You can specify multiple tags. Items are returned only when all specified tags match.

Example command and output

```
$ feast entities list

NAME      DESCRIPTION  TYPE
driver_id driver id    ValueType.INT64
```

7.7. FEATURE STORE FEATURE VIEWS COMMAND

The **feast feature-views list** command displays a list of all registered feature views.

Feature views syntax

```
feast feature-views list [options]
```

Feature views options

--tags <text>

Filters the list by tags (for example, **--tags 'key:value'**). You can specify multiple tags. Items are returned only when all specified tags match.

Feature views example command and output

```
$ feast feature-views list

NAME            ENTITIES  TYPE
driver_hourly_stats {'driver'} FeatureView
```

7.8. FEATURE STORE INIT COMMAND

The **feast init** command creates a new feature repository to store feature definitions.

Init syntax

```
feast init <repository_name> [options]
```

Init options

-t

Specifies a template for the repository (for example, **gcp** or **aws**).

Init examples

- **Create a repository with the default template**

The following command creates a repository named **my_repo_name**:

```
$ feast init my_repo_name
```

Init output

```
Creating a new Feast repository in /projects/my_repo_name.
```

```
.
├── data
│   └── driver_stats.parquet
├── example.py
└── feature_store.yaml
```

- **Create a repository using the Google Cloud Platform (GCP) template:**

```
$ feast init -t gcp my_feature_repo
```

- **Set the name of the new project:**

```
$ feast init -t gcp my_feature_repo
```

7.9. FEATURE STORE MATERIALIZE COMMAND

Use the **feast materialize** command to load data from feature views into the online store for a specific time range.

Materialize syntax

```
$ feast materialize [options] <start_date> <end_date>
```

Options

--disable-event-timestamp

Materializes all available data using the current date and time as the event timestamp. This flag is useful when the source data lacks event timestamp columns.

